



UNIVERSITY OF
CAMBRIDGE

Constrained Decoding for Large Language Models

Test-Time Logit Fusion for
Low-Resource Language Adaptation

A REPORT PRESENTED BY

TIANSHUAI GAO

Department

Department of Physics (Cavendish Laboratory)

Degree

MPhil in Data Intensive Science

Supervision

Dr Weiwei Sun



**SIDNEY SUSSEX COLLEGE
UNIVERSITY OF CAMBRIDGE**

12TH JULY 2026

Word Count: 6967 (excluding references and the declaration of use of auto-generation tools)

Abstract

Large language models serve low-resource languages poorly, and full fine-tuning at scale is costly. Decoding-time logit fusion adapts a frozen large model by mixing its next-token logits with a small language-adapted model. We reproduce two published fusion methods, Proxy Tuning and TriMix, on the MiLiC-Eval benchmark across four low-resource languages of China and three backbone scales. Both methods weaken as the backbone grows, and at 32B they fall below the large model alone. We call this the dilution effect. We propose SAF-W, a training-free fusion of a host and a scorer. The host proposes each token, and the scorer’s endorsement of that token sets the mixing weight. The only configuration is a one-bit host choice per task and language, made on development data. This choice matches the test-best host in 53 of 60 cells. SAF-W runs two models instead of three, and its advantage over TriMix grows from +2.0% at 7B to +9.4% at 32B. The findings transfer to Gemma-3 and to code-switched translation.

List of Figures

- 1 Overview of SAF-W. At each step the host model proposes its most likely token, and the scorer model reports the probability it assigns to that token. This endorsement sets a per-token weight that fuses the two logit distributions. When the scorer endorses the proposal the fusion leans toward the host, when it is neutral the two are averaged uniformly, and when it rejects the proposal the fusion leans toward the scorer. The host role is fixed per task and language on development data. 12
- 2 The better host tracks single-model strength, and this holds across scales. Each point is one task and language cell. The x -axis is the CPT minus instruction single-model gap. The y -axis is the CPT-host minus instruction-host gap. When one model is the stronger standalone system, the better host is the other one (shaded regions). Bands show 95% confidence intervals. 23
- 3 For most tokens where the two models differ, the CPT model is already more accurate (grey lines, 69 to 92%). Keeping only strong vetoes, with w below 0.1, raises this to 95 to 100% for every language, both directions, and title generation. 26
- 4 Each point is a token where the two models differ, for Uyghur into English. Low w falls where the CPT model is much better (upper left). High w falls where the host is better (lower band). The red points are the 3% where the CPT model is better but the host is kept. This residual is the headroom the oracle recovers. . . 27

List of Tables

- 1 The six MiLiC-Eval tasks, their metrics, and capability axis: **ST** = Scaling-Transfer, **LN** = LRL-Native, **R** = Reasoning. Metrics: ROUGE-L (Lin, 2004) and chrF++ (Popović, 2017). 17
- 2 Qwen2.5 results by **capability axis**. #**M** counts the models combined at inference. Overall $-m$ excludes math and $+m$ includes all six tasks. Parentheticals on $-m$ give the relative gain of SAF-W over TriMix and of each three-model baseline over the best single model. Proxy Tuning is not run at 32B. SAF-W deploys the host selected on the devx set (Table 7). *oracle*[†] picks the better host per test example and is an upper bound. **Bold** marks $\text{SAF} \geq \text{TriMix}$. [‡]SAF-W decodes math at $\delta_t=0$ (uniform averaging). Per-language math is in Table 9. 19
- 3 Ablating the endorsement offset at 7B (Qwen). Fixing $\delta_t=0$ reduces SAF-W devx-select to uniform averaging on the same two models. Math is identical by construction and excluded. Left, per-task means over the four languages. Right, five-task averages per language and overall. 20
- 4 Gemma-3 results by **capability axis**. Columns and markers follow Table 2. One Gemma CPT checkpoint per language, all at 4B. Proxy Tuning is not run on Gemma. One TriMix math cell (Mongolian, 27B) is missing, so its affected averages are marked $-$ 21
- 5 CoVoSwitch csw $\rightarrow$ en (200 samples). chrF++ and %T, the share of outputs in English under fastText langid. Hosts are Llama-2-13B for Tamil, Estonian, and Slovenian, and Mixtral-8x7B for Welsh, each paired with the corresponding language CPT model. SAF-W runs ins-host in all cells. Bold marks the best value per column. 22
- 6 CoVoSwitch csw $\rightarrow$ xx (200 samples). chrF++ and %T, the share of outputs in the target language under fastText langid. The target is the code-switched language of each column. SAF-W runs ins-host in all cells. A dash marks runs where the language-identification pass was not logged. Bold marks the best value per column. 22
- 7 SAF-W host scores (%) on the devx set (dev + train₂ + train₃). **Bold** marks the selected host. Math is not shown because SAF-W decodes math with uniform averaging. That mode is symmetric, so no host choice applies. 24
- 8 The eight cells where devx-select gives up points to max. The devx columns repeat the selection scores of Table 7, with the deployed host in **bold**. The test columns give both deployments from Appendix C, and the cost is max minus devx-select. RC scores move in steps of 2.5 points and RS in 1.25, so the thin margins span one or two items. The Mongolian RS tie at 14B deploys the ins host. 25
- 9 Math accuracy (%) across all methods. Uniform averaging is the $\delta_t=0$ case of SAF-W. It outperforms every biased fusion method (Proxy, TriMix, and both SAF-W host assignments) at almost every scale and language, because biased fusion suppresses the reasoning chain (Table 10). This is why SAF-W decodes math at $\delta_t=0$. **Bold** marks uniform averaging. 28

10	Host-trusting fusion suppresses the reasoning chain on math. Uniform averaging ($\delta_t=0$) reads the problem before computing. Both SAF-W and TriMix collapse the chain into bare arithmetic over surface numbers, applying the wrong operation. Since uniform averaging is the $\delta_t=0$ case of SAF-W, the fix lies within our framework, and math alone is decoded at $\delta_t=0$	29
11	H3a against Proxy Tuning at 70B on the benchmarks of Liu et al. (2024) , all with the dynamic gain. The H3a planner is Llama-2-70B in every row, paired with the listed verifier, so H3a runs two models. Proxy Tuning steers the same Llama-2-70B base with the same tuned 7B model as expert and Llama-2-7B as anti-expert, so it runs three models. 70B-chat is the directly tuned model, and 70B base is the untuned planner alone. All reference numbers come from the original paper. Bold marks the better fusion method.	31
12	Scores (%) on Tibetan tasks of MiLiC-Eval (Qwen). Rows are defined at the start of this appendix. Bold marks cells $\geq$ TriMix.	37
13	Scores (%) on Uyghur tasks of MiLiC-Eval (Qwen). Rows are defined at the start of this appendix. Bold marks cells $\geq$ TriMix.	38
14	Scores (%) on Mongolian tasks of MiLiC-Eval (Qwen). Rows are defined at the start of this appendix. Bold marks cells $\geq$ TriMix.	39
15	Scores (%) on Kazakh tasks of MiLiC-Eval (Qwen). Rows are defined at the start of this appendix. Bold marks cells $\geq$ TriMix.	40
16	Scores (%) on Tibetan tasks of MiLiC-Eval (Gemma). The CPT model is gemma-3-4b-bo-cpt. Rows are defined at the start of this appendix. Bold marks cells $\geq$ TriMix.	41
17	Scores (%) on Uyghur tasks of MiLiC-Eval (Gemma). The CPT model is gemma-3-4b-ug-cpt. Rows are defined at the start of this appendix. Bold marks cells $\geq$ TriMix.	41
18	Scores (%) on Mongolian tasks of MiLiC-Eval (Gemma). The CPT model is gemma-3-4b-mn-cpt. Rows are defined at the start of this appendix. The TriMix math cell at 27B is missing, so its six-task averages are marked -. Bold marks cells $\geq$ TriMix.	42
19	Scores (%) on Kazakh tasks of MiLiC-Eval (Gemma). The CPT model is gemma-3-4b-kk-cpt. Rows are defined at the start of this appendix. Bold marks cells $\geq$ TriMix.	42

Contents

1	Introduction	7
2	Background and Related Work	9
2.1	Logit Fusion for Language Adaptation	9
2.2	Parameter-Space Merging	9
2.3	Guided Decoding for Alignment	10
3	Methods	11
3.1	Problem Setup	11
3.2	Reproduced Baselines	11
3.3	The Dilution Effect	12
3.4	SAF-W	12
3.5	Host Selection	14
4	Implementation and Experimental Pipeline	15
4.1	Decoding Implementation	15
4.2	Reproducing the Baselines	15
4.3	High-Performance Computing Setup	16
4.4	Reproducibility and Experiment Management	16
5	Results	17
5.1	Experimental Setup	17
5.2	Main Results	18
5.3	Ablating the Endorsement Offset	19
5.4	Generalizability	20
5.5	Code-switching	21
6	Analysis	22
6.1	Why Host Selection Works	22
6.2	What the Endorsement Weight Encodes	25
6.3	Headroom of Host Selection	27
6.4	Why Math Needs Uniform Averaging	27
7	Bonus: H3a, a Disagreement-Weighted Decoder at 70B	30
8	Conclusion	31
9	Limitations	32
A	README.md File	36

B	Use of Auto-generation Tools	36
B.1	Claude	36
B.2	GitHub Copilot	36
C	Full Per-Language Results	37
C.1	Qwen2.5	37
C.2	Gemma-3	41
	Attachment: README.md	43

1 Introduction

Large language models increasingly operate inside software systems, where their output must satisfy strict requirements that free-form generation does not guarantee. Constrained decoding addresses this by intervening at inference time, reshaping the token distribution at each step so that the output meets a constraint the model would otherwise violate. The most developed form of this idea enforces a formal grammar, restricting generation to outputs that parse as well-formed JSON, executable SQL, or a domain-specific language (Geng et al., 2023). The same principle extends to multilingual settings, where constrained decoding has been used to project annotations across languages and improve transfer to low-resource targets (Le et al., 2024). The constraint signal need not be a symbolic grammar, however. It can also be a second model, whose distribution corrects the first during generation. This report studies the latter form and applies it to a setting where a large model needs knowledge it cannot obtain from its own training, the adaptation of large language models to low-resource languages. This setting also surfaces the tokenization mismatch that constrained decoding must contend with, since a single character in a low-resource script often splits into several subword tokens that the large model handles poorly.

Large language models excel in high-resource languages but remain limited in low-resource languages (LRLs) (Singh et al., 2025; Zhang et al., 2023). Adapting them confronts two constraints. Labeled task data is scarce, because annotation in these languages is costly and few annotators are available. This leaves little supervision for fine-tuning. Continually pretraining or merging at the scale of a large model is also expensive, placing such adaptation beyond the reach of most practitioners working on LRLs (Joshi et al., 2020; Urbizu et al., 2025; Nag et al., 2025). These constraints rule out most weight-update approaches and motivate methods that combine models only at decoding time, fusing the logits of an off-the-shelf large model with those of a small model specialized to the target language, without any further training (Hinton, 2002; Liu et al., 2021).

Two such methods define the current approach. Proxy Tuning (Liu et al., 2024) injects the small model’s adaptation signal into the logits of the large model, and performs well in domains where the large model is already competent. In LRLs, however, the large model often lacks competence in the target language, so letting it dominate the fused logits can overwhelm the small model and degrade generation (Zhang et al., 2026). TriMix (Zhang et al., 2026) addresses this by balancing three sources of ability with weights selected per task. The two methods differ in how they set these weights, yet both rest on a shared assumption that with the right weights, fusing the two models surpasses the large model alone.

This assumption has been tested only on small and mid-size models, where fusion does help. Its benefit narrows as the large model grows stronger, however, and beyond a certain scale it reverses into a loss. A large model that is already competent gains nothing from a weaker model mixed into its logits, and no weighting repairs the resulting dilution. The stronger the large model, the more a fixed mixture costs.

We present **SAF-W**, a decoding-time fusion method built on the opposite principle of trusting one model by default and intervening only where it is clearly wrong. Like prior fusion methods, SAF-W needs no task annotation and no training of a large model, continually pretraining only a small model on raw target-language text. We designate one model the host. At each token the host proposes its most likely word, and we weight the host by the probability the other model

assigns to that word. This weight requires neither training nor tuning. When the host is reliable the other model concurs and the host retains control, and weight shifts to the other model only on the rare words where the host fails. The host role is a single choice per task and language, fixed on a small development set. Because SAF-W leaves a strong host almost untouched, its advantage grows with the large model rather than eroding, reversing the decline that fusion otherwise suffers at scale.

This report makes the following contributions. First, it reproduces two existing logit-fusion methods, Proxy Tuning and TriMix, and evaluates them on the MiLiC-Eval benchmark across four low-resource languages and three model scales, within a single inference and analysis pipeline. Second, it identifies a dilution effect in decoding-time fusion, where these methods lose ground to the single large model as it grows stronger, contradicting the assumption that fusion is uniformly beneficial. Third, it proposes SAF-W, a fusion method that combines a tuning-free per-token gate with a one-bit host choice, and whose advantage grows with the strength of the large model. Fourth, it provides a token-level analysis of why SAF-W works, showing that the gate acts only on the rare words where one model is clearly better, and confirms the method on a second model family and a code-switching benchmark.

The report is organized as follows. Section 2 reviews constrained decoding, logit fusion, and related inference-time methods. Section 3 presents the problem setup, the two reproduced baselines, SAF-W, and the implementation. Section 5 reports results on Qwen2.5 and Gemma-3. Section 6 analyzes why host selection and the endorsement weight work. Section 7 reports a bonus experiment with a second two-model decoder at 70B. Section 8 concludes and discusses limitations.

2 Background and Related Work

Constrained decoding covers any method that intervenes at inference time to make a model’s output satisfy a constraint it would otherwise miss. The two facets named in the project description, the alignment between tokenization and a downstream task and the coherence between incremental token prediction and a target structure, both arise from the same mechanism, a reshaping of the per-step token distribution. Grammar-constrained decoding reshapes it with a symbolic signal, masking tokens that violate a formal grammar so that the output parses (Geng et al., 2023). The work in this report reshapes it with a neural signal, the distribution of a second model, so that generation in a low-resource language stays correct. The two share a setting but differ in the source of the constraint. We review the line of work that uses a second model as the constraint signal, the parameter-space alternative, and related inference-time methods that also let a small model steer a large one.

2.1 Logit Fusion for Language Adaptation

A line of work combines models at decoding time by operating directly on their output distributions. This avoids updating the weights of a large model. The idea originates in the product of experts (Hinton, 2002), where several distributions are multiplied to form a sharper consensus. DExperts (Liu et al., 2021) extends it, steering a base model by adding the logits of an expert and subtracting those of an anti-expert. Proxy Tuning (Liu et al., 2024) applies this arithmetic to adaptation, injecting the difference between a tuned and an untuned small model into the logits of a large model so that the large model inherits the small model’s specialization. For low-resource languages this arithmetic is fragile, because the large model is itself weak in the target language and tends to overwhelm the small model that carries the relevant competence (Zhang et al., 2026). TriMix (Zhang et al., 2026) addresses this by fusing three sources of ability with weights chosen per task, up-weighting the small continually pretrained model where the large model is unreliable. These methods differ in how many models they combine and how they set the weights, yet they share two properties that we revisit. The fusion weights are fixed once chosen for a task, so the same mixture applies to every token, and the methods are validated only up to mid-size large models, where fusion reliably helps. Neither property has been tested at larger scale. There we find that a fixed mixture begins to dilute a capable large model.

2.2 Parameter-Space Merging

A separate family adapts models in weight space rather than at decoding time. Model merging combines the parameters of several independently trained models into one, and has been applied to low-resource languages by merging a continually pretrained model with an instruction-tuned one (Tao et al., 2024). Merging requires the combined models to share an architecture and a scale, so it cannot fuse a small specialized model with a larger general one, and instead forces continual pretraining at the larger scale to obtain scaling benefits (Zhang et al., 2026). It therefore addresses a different setting from ours and is not directly comparable.

2.3 Guided Decoding for Alignment

Closest to our mechanism are inference-time methods that let a small model steer a large one during decoding. NUDGING (Fei et al., 2025) lets a large base model generate and hands control to a small aligned model wherever the base model’s own top-token probability falls below a threshold, and KAD (Hammal et al., 2026) casts the same handoff as a deferral problem solved with a knapsack approximation. Three differences separate these methods from ours. They are cascading methods that assign each token to a single model, whereas SAF-W fuses both distributions at every token through a continuous weight. Their decision to intervene reads only the large model’s own confidence, whereas our endorsement weight reads the second model’s probability for the proposed token and so measures agreement between the two models directly. Finally they target instruction alignment, where a small aligned model adjusts a handful of stylistic tokens and the large model is assumed competent throughout, whereas we target low-resource adaptation, where the large model may lack the language itself and the small model carries knowledge it cannot supply. This last difference is why the host in our setting cannot be fixed to the large model but must be selected per language.

3 Methods

This section presents the decoding-time fusion methods studied in this report. We first set up the fusion problem (§3.1). We then describe the two existing methods we reproduce, Proxy Tuning and TriMix (§3.2), and identify the limitation that motivates our work, the dilution effect (§3.3). We then present **SAF-W**, our method. It keeps uniform averaging as a default and adds a per-token endorsement offset (§3.4). Host selection (§3.5) and the implementation (§4) follow.

3.1 Problem Setup

We combine two frozen models at inference time. The first is a large instruction-tuned model M^L . The second is a small model M^C , obtained by continued pretraining (CPT) on monolingual text in the target low-resource language (LRL). Let x be the prompt and $y_{<t}$ the tokens generated so far. At each step t , both models produce next-token logits over a shared vocabulary V ,

$$\ell_t^L = M^L(x, y_{<t}), \quad \ell_t^C = M^C(x, y_{<t}) \in \mathbb{R}^{|V|}. \quad (1)$$

We assume the two models share a tokenizer, as both come from the same base family.¹ Every method below decodes from a fused logit vector z_t and differs only in how that vector is formed.

3.2 Reproduced Baselines

We reproduce two existing logit-fusion methods and use them as baselines. Both inject the small model’s language ability into the large model’s logits at decoding time, without any training.

Proxy Tuning. Proxy Tuning (Liu et al., 2024) adds the difference between the CPT model and the small base model onto the large model’s logits,

$$z_t = \ell_t^{\text{ins}} + (\ell_t^{\text{cpt}} - \ell_t^{\text{base}}). \quad (2)$$

The difference term is the adaptation signal carried by continued pretraining. Adding it steers the large model toward the target language. Proxy Tuning combines three models, the large instruction model, the small CPT model, and the small base model.

TriMix. TriMix (Zhang et al., 2026) generalizes this with explicit weights on three sources,

$$z_t = \alpha \ell_t^{\text{ins}} + \beta \ell_t^{\text{cpt}} + (1 - \alpha - \beta) \ell_t^{\text{base}}, \quad (3)$$

where the weights (α, β) are selected per task by a perplexity heuristic over a small grid, with no access to labels. We reproduce this selection procedure and report the resulting variant. TriMix also combines three models.

¹Cross-tokenizer fusion is possible through vocabulary alignment but is orthogonal to our contribution. We assume a shared vocabulary throughout, as is standard in the proxy-tuning line of work.

3.3 The Dilution Effect

Both reproduced methods rest on a shared assumption, that with the right weights, fusing the two models surpasses the large model alone. We identify three properties of this fixed-weight design that limit it as the large model grows stronger.

1. **Fixed weights applied to every token.** The weights in Eq. (2) and Eq. (3) are set once per task and applied to every token. The same mixture governs tokens the large model handles well and tokens it does not. The small model is mixed in even where the large model is already correct.
2. **No mechanism to defer to the stronger model.** Neither method reads, at each step, which model is more reliable for the current token. The mixture cannot step back when the large model is confident and correct.
3. **Validated only at small and mid scale.** Prior evaluations stop at mid-size large models, where fusion reliably helps. At larger scale, a large model that is already competent gains nothing from a weaker model mixed into its logits, and no fixed weighting repairs the resulting dilution. We confirm this reversal empirically in §5.

These limitations motivate a method that trusts the large model by default and intervenes only where it is clearly wrong.

3.4 SAF-W

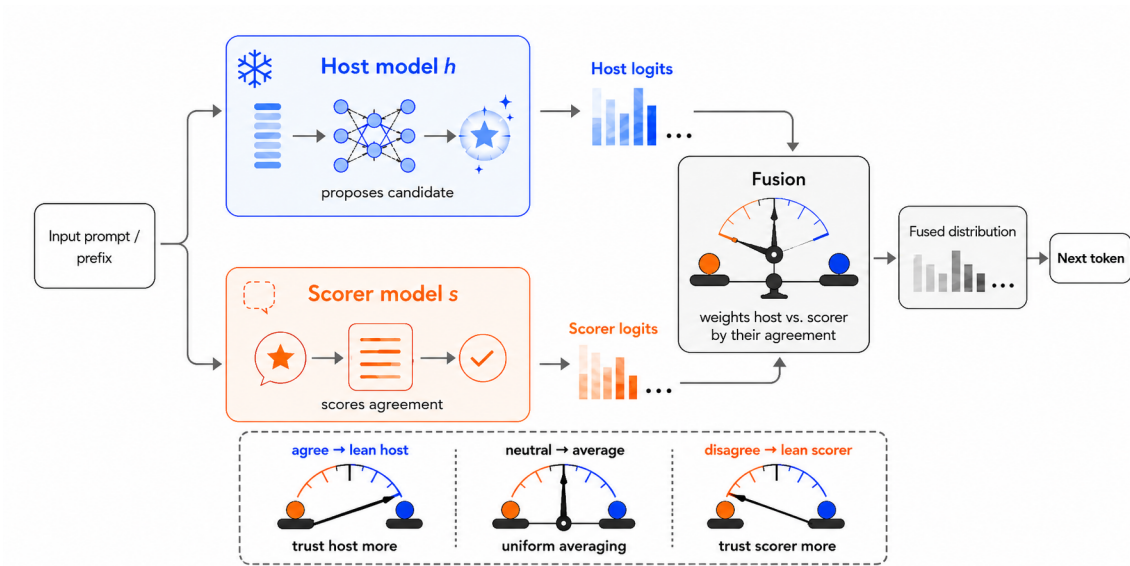


Figure 1. Overview of SAF-W. At each step the host model proposes its most likely token, and the scorer model reports the probability it assigns to that token. This endorsement sets a per-token weight that fuses the two logit distributions. When the scorer endorses the proposal the fusion leans toward the host, when it is neutral the two are averaged uniformly, and when it rejects the proposal the fusion leans toward the scorer. The host role is fixed per task and language on development data.

From uniform averaging to adaptive fusion. The simplest way to combine two models in logit space is to average them,

$$z_t = \frac{1}{2} \ell_t^L + \frac{1}{2} \ell_t^C, \quad (4)$$

which we call **uniform averaging**. After softmax this is an equal-weight product-of-experts (Hinton, 2002). A token survives only when both models give it non-negligible probability. Uniform averaging weights the two models equally at every step, and ignores which model is more reliable for the current token. A per-token coefficient $\beta_t \in [0, 1]$ generalizes this,

$$z_t = (1 - \beta_t) \ell_t^L + \beta_t \ell_t^C, \quad (5)$$

and we decode from $\text{softmax}(z_t)$. A fixed $\beta_t \equiv \frac{1}{2}$ gives uniform averaging. A fixed 0 or 1 gives one model alone. Any other fixed value gives a non-adaptive linear ensemble. SAF-W keeps uniform averaging as the default, then adds a per-token, parameter-free offset that moves the mixture toward whichever model is more trustworthy at that step.

Endorsement offset. We call one model the **host** h and the other the **scorer** s , with $\{h, s\} = \{L, C\}$. The assignment is fixed per task and language (§3.5). At each step, the host proposes its most likely token,

$$v_t = \arg \max_{v \in V} \ell_t^h[v], \quad (6)$$

and the scorer reports the probability it gives to that token,

$$e_t = p_t^s[v_t] = \text{softmax}(\ell_t^s)[v_t] \in [0, 1]. \quad (7)$$

We call e_t the **endorsement**. We turn it into a signed offset around the uniform-averaging midpoint,

$$\delta_t = e_t - \frac{1}{2} \in [-\frac{1}{2}, \frac{1}{2}], \quad (8)$$

and fuse the two models as uniform averaging shifted by δ_t toward the host,

$$z_t = \left(\frac{1}{2} + \delta_t\right) \ell_t^h + \left(\frac{1}{2} - \delta_t\right) \ell_t^s. \quad (9)$$

The offset has three regimes. When the scorer is neutral ($e_t = \frac{1}{2}$, $\delta_t = 0$), the fusion is exactly uniform averaging (Eq. (4)). When the scorer endorses the proposal ($e_t \rightarrow 1$), weight moves to the host. When the scorer rejects it ($e_t \rightarrow 0$), weight moves to the scorer.

Computation. Eq. (9) makes the symmetric midpoint explicit. The constant $\frac{1}{2}$ cancels once we substitute $\delta_t = e_t - \frac{1}{2}$. This leaves a one-line update,

$$z_t = e_t \ell_t^h + (1 - e_t) \ell_t^s. \quad (10)$$

The host is weighted by exactly the probability the scorer gives to its proposal. This is what we run at decoding time. Each step costs one $\arg \max$, one gather, and one convex combination, with no tuning. The two forms are algebraically identical. Eq. (9) exposes the offset δ_t . We study it as a per-task design axis in §6. Eq. (10) is its computational form. With the instruction model

as host, $\beta_t = 1 - e_t$ in Eq. (5), and the CPT-host case swaps the roles. We recompute the offset at every token from the scorer’s probability on the proposed token, and never fix it once from a prompt-level statistic.

3.5 Host Selection

The two assignments $h = L$ and $h = C$ give different decoders. The offset shifts uniform averaging toward the host, and the scorer stays a corrective. SAF-W treats the host as one discrete hyperparameter, selected per task and language on held-out annotated data outside the prompt exemplars, choosing the value that maximizes the selection metric. SAF-W has no continuous parameters. Each setting is fixed by a few discrete bits, all chosen on development data. The host is chosen on development data and evaluated on test, so the decoder is directly deployable. We study the effect of this choice, and the gap to a per-example oracle, in §6.

A second bit, also chosen per task and language, picks between the endorsement offset and fixing $\delta_t \equiv 0$ (uniform averaging, Eq. (4)). We select it on development data, like the host. Every task except mathematical reasoning selects the endorsement offset. Only math selects uniform averaging. The reason is that the offset suppresses the step-by-step chain that math needs. We show this in §6.4.

Inference cost. Each step runs one forward pass of M^L and one of M^C . The endorsement (Eq. (7)) is an $\arg \max$ and a gather. The fusion (Eq. (10)) is a vector combination. Both are negligible against the forward passes. The per-token cost is $C_L + C_C$, and $C_L \gg C_C$ because the CPT model is small. The first step has no proposal to score yet, so it falls back to uniform averaging ($\delta_0 = 0$). Proxy Tuning and TriMix instead combine three models, a base, an expert, and a separately trained anti-expert, and run a third forward pass at every step. SAF-W uses two models. It removes both the third forward pass and the anti-expert that pass requires.

4 Implementation and Experimental Pipeline

This section describes the code that produces our results. We first describe the decoding implementation and the framework it extends (§4.1). We then describe how we reproduce the two baselines (§4.2). We cover the cluster setup in §4.3 and the steps we take for reproducibility in §4.4. Full installation and run instructions are given in Appendix A.

4.1 Decoding Implementation

All four decoders share one implementation. A base decoder runs a cached greedy generation loop over a set of frozen models. Each method is a subclass that overrides only the fusion rule. Proxy Tuning and TriMix use their three-model rules from §3.2. SAF-W uses the endorsement fusion of Eq. (10). The generation loop, the prompt construction, and the evaluation code are shared across all methods. This keeps the comparison controlled, because every method runs through the same code path except for one function.

We build SAF-W on the released TriMix decoder. SAF-W is a fork of that framework with a new fusion subclass added. We do this so that the baselines are reproduced faithfully and the only moving part is the fusion rule. We keep the full decoding machinery of the original framework rather than write a simplified loop. We retain the key-value cache, so each step attends to cached states and only the new token is processed. We retain batched generation with per-sequence stopping, so a batch can finish its sequences at different steps. We retain the stopping criteria, the vocabulary alignment between models of different vocabulary size, and the chat formatting path. Faithful reproduction matters here, because the thesis goal is to reproduce the baselines as they were published and to measure SAF-W against them on equal terms. A simplified reimplementation could change the baseline numbers and break that comparison.

The per-token cost of SAF-W is small on top of the forward passes. Each step computes one argmax over the host logits, one gather of the scorer probability at the proposed token, and one convex combination of the two logit vectors. These operations are negligible against the two forward passes. SAF-W runs two models and so runs two forward passes per step. Proxy Tuning and TriMix run three models and so run a third forward pass at every step.

4.2 Reproducing the Baselines

We reproduce Proxy Tuning and TriMix in the same framework as SAF-W. The three methods share the generation loop, the prompts, and the evaluation, so the baselines run under the same conditions as our method. The only difference is the fusion rule and the number of models.

Proxy Tuning adds the expert and anti-expert residual to the base logits, following Eq. (2). TriMix takes the weighted three-model combination of Eq. (3). TriMix also applies a plausibility constraint that masks tokens the plausibility model finds unlikely. We keep this constraint in our reproduction.

TriMix selects its weights per task and language. We reproduce this selection procedure. We sweep a grid of weight values on a small development set and compute the perplexity of each combination. We then pick the combination with the lowest perplexity. This sweep uses no labels. It runs as a separate script before the main inference. We report the variant selected by this

procedure.

4.3 High-Performance Computing Setup

We run all experiments on the Cambridge CSD3 cluster. We use the Ampere GPU partition through the SLURM scheduler. We load every model in bfloat16. We run greedy decoding throughout, so there is no sampling and no temperature.

We submit experiments with a batch script. The script wraps a single driver. A run is declared with the method, the language, the scale, and for SAF-W the host side. The driver resolves the model paths from these declarations and checks every declaration against the resolved names. A mismatch stops the job before any model loads. One submission runs all six tasks for one language at one scale. The baselines run through the same driver by declaring their method.

The cluster places a wall-clock limit on each job. The translation and title tasks generate long outputs and are the slowest. We give each job a wall-clock budget that covers the full set of tasks for one language at one scale.

4.4 Reproducibility and Experiment Management

We take several steps so that every run is reproducible and no run overwrites another.

Each configuration writes to its own output file. The file name encodes the method, the model family, the language, the task, the scale, and the host. Two runs with different settings therefore write to different files and cannot overwrite each other. This matters because we run many configurations across scales, languages, tasks, and host assignments, and a shared name would silently lose data.

The few-shot exemplars are fixed, so every run uses the same exemplars in the same order. We set the CUDA workspace configuration to make the matrix operations deterministic on the Ampere GPUs. With these settings greedy decoding is reproducible across runs.

We separate prediction files from metric files. Inference writes one prediction file per configuration. Evaluation reads that file and writes a metric file beside it. The prediction file holds the generated text for every example, so we can re-score it later without re-running inference. Full installation steps, the run commands for every method, and the cluster settings are given in Appendix A.

5 Results

5.1 Experimental Setup

Models. We primarily evaluate on the Qwen2.5 family (Qwen Team, 2024), pairing a 1.5B continually pretrained (CPT) model with instruction-tuned models at 7B, 14B, and 32B. To test generality across architectures, we also evaluate Gemma-3 (Gemma Team, 2025), pairing a 4B CPT model with 12B and 27B instruction-tuned models. The CPT models and their pretraining corpora follow Zhang et al. (2026), and we use their released checkpoints throughout.

Languages. We evaluate on four low-resource languages of China with distinct scripts. These are Tibetan, Uyghur, Kazakh, and Mongolian.

Evaluation. We use the MiLiC-Eval benchmark (Zhang et al., 2025) and evaluate on six of its tasks, with metrics and grouping summarized in Table 1.² We organize the tasks along three capability axes. The large model leads on Scaling-Transfer, the CPT model leads on LRL-Native, and Reasoning isolates multi-step math. This grouping mirrors the host-selection decision in SAF-W (§3.5).

Task	Metric	Axis
Reading comprehension	Acc.	ST
Response selection	Acc.	ST
Dialogue MT (xx→en)	chrF++	ST
Title generation	ROUGE-L	LN
Dialogue MT (en→xx)	chrF++	LN
Math (English CoT)	Acc.	R

Table 1. The six MiLiC-Eval tasks, their metrics, and capability axis: **ST** = Scaling-Transfer, **LN** = LRL-Native, **R** = Reasoning. Metrics: ROUGE-L (Lin, 2004) and chrF++ (Popović, 2017).

Code-switching. As an additional robustness check beyond the Chinese minority languages, we evaluate on CoVoSwitch (Kang, 2024), a code-switched translation benchmark, using language-specific model pairs. Details are in §5.5.

Baselines. We compare against single models and the two reproduced fusion methods, all sharing the same CPT checkpoint. The single models are the large instruction-tuned model and the small CPT model, each decoded on its own. Proxy Tuning (Liu et al., 2024) and TriMix (Zhang et al., 2026) are the two fusion baselines, defined in §3.2. We report the strongest TriMix variant, with its weights selected per task by the perplexity heuristic. SAF-W combines two models, while Proxy Tuning and TriMix combine three. Uniform averaging is the $\delta_t = 0$ case of SAF-W (§3.4), so we report it as an ablation rather than as a separate baseline.

²We omit topic classification, whose evaluation we found unstable across runs.

Host selection. SAF-W selects its host per task and language before touching the test set. The dev split alone is small, with 20 items for RC and 40 for the rest, so a single flipped item can change the choice. We therefore select on dev, train₂, and train₃ together and call this set devx. It contains all annotated data outside the prompt exemplars. Table 7 in §6.1 reports both host scores, and the bold host is deployed. Math is excluded because it decodes at $\delta_t = 0$, so no host choice applies.

Oracle. We also report an oracle upper bound to measure the headroom available to host selection. For each test example, the oracle runs SAF-W under both host assignments, instruction-host and CPT-host, and keeps whichever output scores higher against the reference. This choice consults the gold reference of each example, so the oracle is not a deployable method. It measures how much a perfect per-example host router could gain. Our development-set selection (§3.5) is measured against this bound. The oracle uses the same two models and gating rule as SAF-W. It differs only in that the host is chosen with knowledge of the test label.

Implementation. We follow [Zhang et al. \(2026\)](#) for all shared settings, including prompts and the number of in-context demonstrations. SAF-W selects its host on the devx set defined above. Baselines with per-task hyperparameters use the same development budget.

5.2 Main Results

Table 2 reports results on Qwen2.5, grouped by capability axis.

SAF-W beats three-model fusion with one fewer model. SAF-W achieves the best overall score at every scale, using two models against the three of Proxy Tuning and TriMix. It improves over TriMix by 0.6, 2.1, and 3.1 points overall at 7B, 14B, and 32B. The gain is largest on Scaling-Transfer, reaching +4.0 at 32B, where the large model’s task competence is otherwise diluted. SAF-W stays competitive on LRL-Native generation. Removing the anti-expert costs no quality and uses one fewer model.

The advantage grows with scale. The relative gain over TriMix grows with model size, from +2.0% at 7B to +6.5% at 14B and +9.4% at 32B. TriMix improves over the best single model at 7B and 14B, but drops below it at 32B, at −3.8%. A stronger large model is dragged down more by a third weaker model. SAF-W instead tracks the large model upward, because its gate defers to that model wherever the two agree. Its benefit grows with stronger backbones rather than eroding.

The endorsement offset earns its keep. Ablating the offset costs 2.2 points on the five-task average at 7B (§5.3). Uniform averaging alone only reaches the level of the best single model, while Proxy Tuning, despite using a third model, falls below even that on most languages. A parameter-free two-model average already matches or beats the three-model anti-expert scheme. The anti-expert therefore hurts here. The offset then turns the same two models into a further consistent gain.

Math is the exception. Math is the only task where SAF-W does not use the endorsement offset, decoding instead at its $\delta_t=0$ reduction. This choice is made on development data, with no test labels, and we analyze why the offset fails on math in §6.4.

Method	#M	Scaling-Transfer					LRL-Native					Reasoning	Overall	
		rc, rs, xx2en					title, en2xx					math [†]	−m	+m
		Tib	Uig	Mon	Kaz	Avg	Tib	Uig	Mon	Kaz	Avg	Avg		
Reference single models (small = 1.5B)														
Qwen2.5-1.5B-cpt	1	27.7	49.1	26.5	41.3	36.1	22.1	18.4	13.6	17.7	18.0	14.4	28.9	26.5
large = 7B														
Qwen2.5-7B-ins [†]	1	32.4	43.6	32.2	37.8	36.5	22.4	15.8	7.8	8.8	13.7	17.0	27.4	25.7
Proxy Tuning	3	33.3	45.7	31.8	39.5	37.6	23.0	14.6	5.0	10.0	13.2	18.8	27.8	26.3
TriMix	3	30.5	51.2	29.8	44.4	39.0	22.4	18.4	13.4	18.2	18.1	19.1	30.6(+6.0%)	28.7
SAF-W (ours)	2	32.7	50.5	33.1	42.6	39.7	22.3	19.6	13.3	18.1	18.3	25.0	31.2(+2.0%)	30.1
Δ SAF−TriMix		+2.2	−0.7	+3.3	−1.8	+0.7	−0.1	+1.2	0.0	−0.1	+0.2	+5.9	+0.6	+1.4
oracle [†]	2	37.8	53.3	36.4	47.0	43.6	26.5	21.6	15.1	20.0	20.8	29.0	34.5	33.6
large = 14B														
Qwen2.5-14B-ins [†]	1	38.3	51.0	36.8	43.4	42.4	26.1	19.1	7.1	11.2	15.9	8.9	31.8	28.0
Proxy Tuning	3	38.2	52.4	35.9	46.3	43.2	23.1	18.3	9.7	15.3	16.6	19.5	32.6(+2.5%)	30.4
TriMix	3	32.7	54.0	31.3	47.4	41.3	23.0	20.3	13.9	18.1	18.8	18.4	32.3(+1.8%)	30.0
SAF-W (ours)	2	38.1	55.4	34.9	48.2	44.2	25.5	22.1	13.1	18.6	19.8	31.6	34.4(+6.5%)	34.0
Δ SAF−TriMix		+5.3	+1.4	+3.6	+0.8	+2.9	+2.5	+1.8	−0.8	+0.6	+1.0	+13.2	+2.1	+4.0
oracle [†]	2	42.4	59.1	38.8	51.5	47.9	28.9	25.1	15.3	21.6	22.7	27.1	37.8	36.0
large = 32B														
Qwen2.5-32B-ins [†]	1	40.4	54.9	37.1	46.4	44.7	25.4	21.4	11.9	16.9	18.9	2.9	34.4	29.2
TriMix	3	31.7	58.2	31.7	48.3	42.5	23.4	19.8	14.1	18.3	18.9	18.1	33.0(−3.8%)	30.5
SAF-W (ours)	2	40.4	59.7	37.0	49.0	46.5	26.1	22.9	14.2	18.5	20.4	26.9	36.1(+9.4%)	34.5
Δ SAF−TriMix		+8.6	+1.4	+5.3	+0.7	+4.0	+2.7	+3.1	+0.2	+0.2	+1.5	+8.8	+3.1	+4.0
oracle [†]	2	43.7	62.4	39.1	52.2	49.4	29.2	25.7	15.9	23.6	23.6	20.0	39.1	35.9

Table 2. Qwen2.5 results by **capability axis**. #M counts the models combined at inference. Overall −m excludes math and +m includes all six tasks. Parentheticals on −m give the relative gain of SAF-W over TriMix and of each three-model baseline over the best single model. Proxy Tuning is not run at 32B. SAF-W deploys the host selected on the devx set (Table 7). oracle[†] picks the better host per test example and is an upper bound. **Bold** marks SAF $\geq$ TriMix. [†]SAF-W decodes math at $\delta_t=0$ (uniform averaging). Per-language math is in Table 9.

5.3 Ablating the Endorsement Offset

Fixing $\delta_t \equiv 0$ removes the endorsement offset. The deployed decoder then reduces to uniform averaging on the same two models. The fusion becomes symmetric, so the host role disappears with the offset. Math already decodes at $\delta_t=0$ and is identical by construction, so the comparison covers the five offset tasks. We ran the full ablation row at 7B, and the larger scales were not re-run under this ablation before the cluster outage. Table 3 reports the 7B comparison. The

per-language tables in Appendix C carry the uniform-averaging rows at every scale, and the offset gain holds there as well. The offset adds +2.2 points on the five-task average, and every language gains, from +0.6 on Tibetan to +3.4 on Kazakh. The gains concentrate in translation, at +4.5 into English and +3.5 into the low-resource language. The classification gains are small. Title generation is mixed, and on Tibetan the symmetric mixture beats both host assignments. The translation gains match the token-level analysis above. The weight needs long outputs to act, and translation supplies them.

Method	Per task					Per language				Overall
	RC	RS	Title	xx2en	en2xx	Tib	Uig	Mon	Kaz	
Uniform averaging ($\delta_t=0$)	50.6	41.8	21.8	19.3	11.2	27.9	34.9	23.5	29.4	28.9
SAF-W devx-select	51.8	43.6	22.0	23.8	14.7	28.5	38.1	25.2	32.8	31.2
Δ offset	+1.1	+1.8	+0.1	+4.5	+3.5	+0.6	+3.2	+1.7	+3.4	+2.2

Table 3. Ablating the endorsement offset at 7B (Qwen). Fixing $\delta_t=0$ reduces SAF-W devx-select to uniform averaging on the same two models. Math is identical by construction and excluded. Left, per-task means over the four languages. Right, five-task averages per language and overall.

5.4 Generalizability

To test whether SAF-W transfers across architectures, we evaluate the Gemma-3 family. We pair a 4B CPT model with 12B and 27B instruction-tuned models and keep the tasks and prompts unchanged. Table 4 reports results by capability axis, and full per-language results are in Appendix C.2. This family probes a different regime, because the 4B CPT model is strong. On Kazakh and Mongolian it beats the 12B instruction model on its own.

The advantage transfers. SAF-W beats the five-task average of TriMix in every language, at both scales and with one fewer model. The margin is 0.9 to 2.4 points at 12B and grows to 1.1 to 2.9 points at 27B, so the scaling trend from Qwen carries over. The gains sit where the Qwen analysis predicts them. On translation into English SAF-W beats TriMix in every language at both scales, by 3.7 to 9.3 points. On math SAF-W decodes at uniform averaging and preserves the instruction model’s reasoning, but the fixed TriMix mixture dilutes it. On Uyghur at 12B the instruction model alone scores 71.5, SAF-W keeps 70.0, and TriMix drops to 54.0.

Native-language generation converges. On title generation both single models collapse and both fusion methods recover the task. On translation into the low-resource language the strong CPT model sets the level, and the two fusion methods land near its score and within one point of each other. The Mongolian TriMix comparison is partial at 27B, because its math cell is missing and the affected averages are marked –. The oracle rows show the same per-example headroom as on Qwen, between 1.8 and 4.3 points on the five-task average.

Method	#M	Scaling-Transfer					LRL-Native					Reasoning	Overall	
		rc, rs, xx2en					title, en2xx					math [‡]	−m	+m
		Tib	Uig	Mon	Kaz	Avg	Tib	Uig	Mon	Kaz	Avg	Avg		
<i>Reference single models (small = 4B, one CPT checkpoint per language)</i>														
Gemma-3-4B-cpt	1	54.8	58.1	34.4	57.1	51.1	22.3	20.9	7.6	19.2	17.5	35.1	37.7	37.2
large = 12B														
Gemma-3-12B-it [†]	1	52.4	67.5	33.1	61.2	53.6	14.7	15.1	8.2	9.8	11.9	45.0	36.9	38.3
TriMix	3	57.9	65.7	33.2	62.0	54.7	43.5	29.6	14.5	28.8	29.1	42.1	44.4 (+17.8%)	44.1
SAF-W (ours)	2	59.7	68.9	36.1	64.1	57.2	43.1	30.6	15.2	28.1	29.2	54.4	46.0 (+3.6%)	47.4
Δ SAF−TriMix		+1.8	+3.2	+2.9	+2.1	+2.5	−0.4	+1.0	+0.7	−0.7	+0.1	+12.3	+1.6	+3.3
oracle [†]	2	61.4	72.3	40.1	66.7	60.1	44.8	34.1	18.5	31.1	32.1	60.1	48.9	50.8
large = 27B														
Gemma-3-27B-it [†]	1	56.3	70.8	34.5	67.1	57.2	15.0	18.7	9.6	13.3	14.1	28.8	40.0	38.1
TriMix	3	58.8	68.5	33.3	65.6	56.6	43.6	29.8	15.2	28.4	29.2	−	45.6 (+14.0%)	−
SAF-W (ours)	2	61.3	72.2	37.1	67.7	59.6	43.0	30.7	15.8	28.0	29.3	43.9	48.5 (+6.4%)	46.9
Δ SAF−TriMix		+2.5	+3.7	+3.8	+2.1	+3.0	−0.6	+0.9	+0.6	−0.4	+0.1	−	+2.9	−
oracle [†]	2	63.8	74.9	42.0	70.4	62.8	44.8	33.5	19.2	31.9	32.4	51.6	50.6	50.8

Table 4. Gemma-3 results by **capability axis**. Columns and markers follow Table 2. One Gemma CPT checkpoint per language, all at 4B. Proxy Tuning is not run on Gemma. One TriMix math cell (Mongolian, 27B) is missing, so its affected averages are marked −.

5.5 Code-switching

Code-switched input stresses the fusion in a way the main benchmark does not. The source mixes English with spans of another language, so neither model owns the input, and the right expert can change within one sentence. We evaluate on CoVoSwitch (Kang, 2024) in both directions, translating code-switched input into English and into the switched language, with 200 samples per direction for Tamil, Welsh, Estonian, and Slovenian. The pairs also move beyond the Qwen and Gemma families. The host is Llama-2-13B for Tamil, Estonian, and Slovenian and Mixtral-8x7B for Welsh, each paired with the CPT model of its language. SAF-W runs ins-host throughout. We did not run the cpt-host direction on this benchmark, so the host-selection protocol of §3.5 is not exercised here. We report chrF++ and the share of outputs in the required target language under fastText language identification, written %T. TriMix is the fusion baseline. Tables 5 and 6 report both directions.

Into English, SAF-W has the best chrF++ on three of the four languages and sits 0.5 behind the instruction model on Slovenian. Tamil shows the sharpest contrast. The Tamil CPT model rarely leaves Tamil, with only 30% of its outputs in English, and its chrF++ collapses to 27.5. TriMix inherits this failure at 28.7. SAF-W keeps the host in charge, holds 82% English output, and still gains 4.8 points over the host alone. A fixed mixture propagates a broken partner. The endorsement contains it.

Into the switched language, the four methods sit close on chrF++. SAF-W leads on Estonian, trails the CPT model by 0.2 on Tamil, and stays within 0.8 of the best on Welsh and Slovenian. Its

target-language share is the highest in three of the four columns, at 99.0, 99.5, and 94.5%. Fusion in this direction mainly repairs language fidelity. The single models drift into the wrong language on part of the inputs, and the per-token weight pulls the output back to the target. A cpt-host run could plausibly help this direction, because the target language is the CPT model’s own, and we leave it untested.

The pattern of the main experiments therefore survives three changes at once. The languages are new, the input is mixed, and the host families are new, with Llama-2 and Mixtral joining Qwen and Gemma.

Method	Tamil		Welsh		Estonian		Slovenian	
	chrF++	%T	chrF++	%T	chrF++	%T	chrF++	%T
ins	54.9	75	61.3	82	59.4	93.5	73.4	95.5
cpt	27.5	30	58.7	80	58.7	82.5	69.2	99.5
TriMix	28.7	30	49.6	87	63.3	89.0	70.4	94.0
SAF-W (ins-host)	59.7	82	65.1	81	68.6	93.5	72.9	96.0

Table 5. CoVoSwitch csw→en (200 samples). chrF++ and %T, the share of outputs in English under fastText langid. Hosts are Llama-2-13B for Tamil, Estonian, and Slovenian, and Mixtral-8x7B for Welsh, each paired with the corresponding language CPT model. SAF-W runs ins-host in all cells. Bold marks the best value per column.

Method	Tamil		Welsh		Estonian		Slovenian	
	chrF++	%T	chrF++	%T	chrF++	%T	chrF++	%T
ins	44.3	–	47.6	96	45.1	–	50.8	–
cpt	59.7	–	46.4	68	53.0	–	48.2	–
TriMix	58.8	97.0	54.7	88	53.8	97.5	49.8	93.0
SAF-W (ins-host)	59.5	99.0	53.9	87	54.2	99.5	50.5	94.5

Table 6. CoVoSwitch csw→xx (200 samples). chrF++ and %T, the share of outputs in the target language under fastText langid. The target is the code-switched language of each column. SAF-W runs ins-host in all cells. A dash marks runs where the language-identification pass was not logged. Bold marks the best value per column.

6 Analysis

6.1 Why Host Selection Works

SAF-W selects a host model for each task and language. We analyze what this selection depends on. The analysis here uses test labels and is post-hoc. The method itself selects the host on a development set, as described in §3.5.

Host choice tracks single-model strength. For each cell we measure two gaps. The first is the gap between the two models run on their own. The second is the gap between the two host settings. These two gaps are strongly anti-correlated across all 72 cells, with $r=-0.81$. The pattern holds at every scale, with $r=-0.85$, -0.76 , and -0.83 at 7B, 14B, and 32B (Figure 2).

When one model is stronger on its own, the other model is the better host. The selection therefore depends only on which model is stronger on its own. This quantity needs no test labels, so a development set is enough to estimate it.

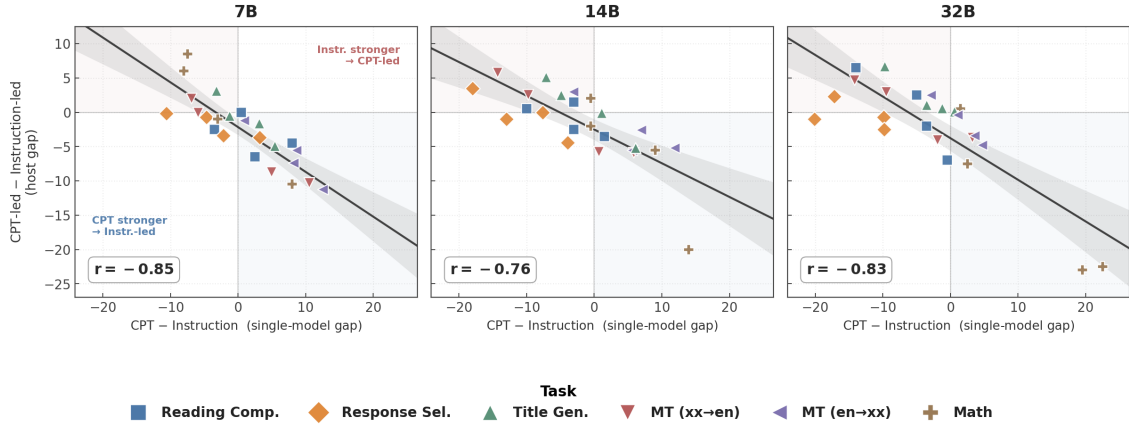


Figure 2. The better host tracks single-model strength, and this holds across scales. Each point is one task and language cell. The x -axis is the CPT minus instruction single-model gap. The y -axis is the CPT-host minus instruction-host gap. When one model is the stronger standalone system, the better host is the other one (shaded regions). Bands show 95% confidence intervals.

Host choice follows task type. Instruction-h decoding wins on 45 of 72 cells. These are reading comprehension, response selection, translation, and reasoning. All four rely on the large model. Title generation is the only task where CPT-host wins more often, because fluent low-resource text needs the language-adapted model. This matches the capability axes in §5.2.

Selection transfers from a development set. We verify this directly. The devx choice (§5.1) matches the test-best host in 53 of 60 cells. Three of the seven misses are Kazakh cells. Kazakh shows a train/test distribution shift in all four tasks, so no selection set repairs these cells. The remaining misses have small test margins, so the deployed host loses little.

Lang	Scale	RC		RS		Title		xx2en		en2xx	
		ins	cpt	ins	cpt	ins	cpt	ins	cpt	ins	cpt
Tib.	7B	37.5	40.0	43.8	37.5	22.1	26.4	7.1	14.1	15.9	15.0
	14B	40.0	50.0	35.0	35.0	25.9	28.9	7.7	22.2	15.7	19.4
	32B	40.0	52.5	43.8	40.0	23.6	29.9	8.4	21.7	16.7	19.4
Uyg.	7B	37.5	35.0	50.0	47.5	17.7	24.2	35.6	30.0	18.1	11.6
	14B	57.5	50.0	58.8	60.0	18.9	26.7	36.0	30.9	17.6	15.0
	32B	60.0	55.0	66.2	63.7	19.8	28.0	35.2	34.4	19.9	17.4
Mon.	7B	40.0	40.0	45.0	47.5	14.7	14.0	6.5	16.0	12.0	5.2
	14B	40.0	42.5	48.8	48.8	16.2	15.4	6.1	15.8	12.1	1.9
	32B	37.5	45.0	47.5	51.2	16.1	16.4	6.9	16.3	12.9	8.3
Kaz.	7B	42.5	50.0	53.8	47.5	20.9	19.9	25.9	22.6	12.8	0.3
	14B	45.0	45.0	56.2	53.8	20.7	20.7	26.2	25.3	12.6	5.8
	32B	47.5	57.5	60.0	60.0	19.9	22.0	28.5	26.9	11.7	13.4

Table 7. SAF-W host scores (%) on the devx set (dev + train₂ + train₃). **Bold** marks the selected host. Math is not shown because SAF-W decodes math with uniform averaging. That mode is symmetric, so no host choice applies.

Where selection costs, and why. The transfer rate counts decisions, and the max row of Appendix C prices them. The cost of a cell is max minus devx-select, the points the deployed host leaves on the table. Table 8 lists every cell with a nonzero cost. There are eight, totalling 16.6 points over 60 cells, an average of 0.28 per cell, and they split into two mechanisms. Four cells are thin-margin flips on the choice tasks. Their devx gaps are one or two items, one is an exact tie, and quantized metrics cannot resolve differences this small. More selection data would. The three Kazakh cells are different. Their devx margins are wide, up to four items, yet the test set reverses them. This is the Kazakh train/test shift noted above, and no selection set repairs it. The remaining title cell costs half a point on a continuous metric. The cost also shrinks with scale, from 9.7 points at 7B to 2.4 at 32B, so selection is most reliable where SAF-W gains most.

Cell		devx score		Deployed	test score		Cost
		ins-host	cpt-host		ins-host	cpt-host	
Kaz. RC	7B	42.5	50.0	cpt	60.5	54.0	6.5
Mon. RS	14B	48.8	48.8	ins	41.5	45.0	3.5
Tib. RC	7B	37.5	40.0	cpt	49.0	46.5	2.5
Kaz. RC	32B	47.5	57.5	cpt	61.0	59.0	2.0
Uig. RS	14B	58.8	60.0	cpt	59.7	58.7	1.0
Tib. Title	7B	22.1	26.4	cpt	29.3	28.8	0.5
Kaz. en2xx	32B	11.7	13.4	cpt	13.2	12.8	0.4
Mon. RS	7B	45.0	47.5	cpt	41.5	41.3	0.2
Total							16.6

Table 8. The eight cells where devx-select gives up points to max. The devx columns repeat the selection scores of Table 7, with the deployed host in **bold**. The test columns give both deployments from Appendix C, and the cost is max minus devx-select. RC scores move in steps of 2.5 points and RS in 1.25, so the thin margins span one or two items. The Mongolian RS tie at 14B deploys the ins host.

6.2 What the Endorsement Weight Encodes

SAF-W computes the per-token weight from one quantity. The host proposes its top token, and the scorer returns the probability it assigns to that token. We denote this $w = p^{\text{cpt}}(v^{\text{ins}})$, and the CPT logits receive weight $1 - w$. We analyze whether w carries useful information.

Setup. For each token we record the host and scorer negative log-likelihoods. We label the scorer as more accurate when its NLL is lower. One case is uninformative. When both models are confident and both NLLs are near zero, the lower one is noise. This is common in Mongolian, where a single character splits into several byte-level tokens that both models predict easily. We therefore restrict the analysis to tokens where the two NLLs differ by more than 0.5.

The weight predicts which model is correct. On these tokens, w is a strong predictor. Using $1 - w$ as a score for the CPT model being more accurate, the AUC is 0.94 for Uyghur and 0.77 to 0.82 for the other languages into English, and 0.84 to 0.87 into the low-resource language. The strong vetoes are the clearest case. When w is below 0.1, the CPT model is more accurate 95 to 100% of the time, across every language, both directions, and title generation (Figure 3). SAF-W therefore intervenes on the tokens where one model is clearly better, and stays neutral elsewhere.

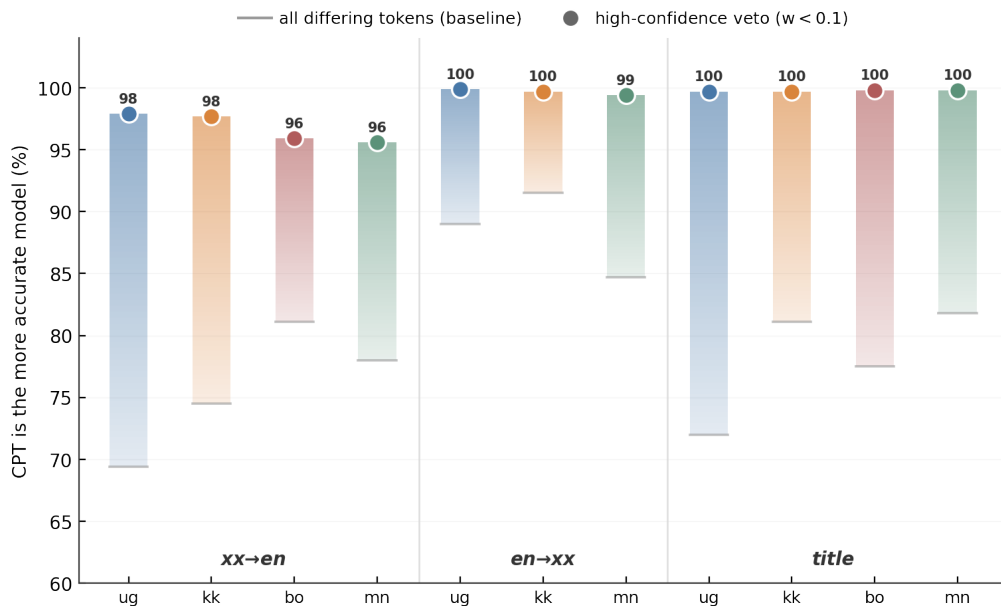


Figure 3. For most tokens where the two models differ, the CPT model is already more accurate (grey lines, 69 to 92%). Keeping only strong vetoes, with w below 0.1, raises this to 95 to 100% for every language, both directions, and title generation.

The low-weight tokens are content words. These tokens carry the content of the translation. There the host is weakest. When translating a Uyghur sentence into English, the host is uncertain on words like *wife*, *skill*, and *garbage*, with NLL above 5. The CPT model has seen Uyghur and is confident, with NLL below 0.5. The scorer assigns these tokens to the CPT model.

The scorer drives the calibration, not the host. The calibration depends on the scorer rather than the host size. For the strong Uyghur CPT model, the AUC is 0.939, 0.929, and 0.934 at 7B, 14B, and 32B. A larger host does not change the routing. This matches the host result in §6.1, where the scoring model drives the outcome. For the weak Tibetan CPT model, the AUC drops from 0.81 to 0.73 as the host grows, because a stronger host narrows the gap the scorer reads.

The weight misses some tokens. Some tokens have the CPT model clearly better, but w is high and the host wins (Figure 4). They are content words such as *newspaper*, *singing*, and *do*. The cause is structural. The weight measures whether the scorer accepts the host token, not whether the scorer has a better one. When the CPT model assigns the host token a reasonable probability, w stays high, even when the CPT model prefers a different token. These tokens are 3% of the cases where the CPT model is clearly better. This residual is the headroom we discuss next.

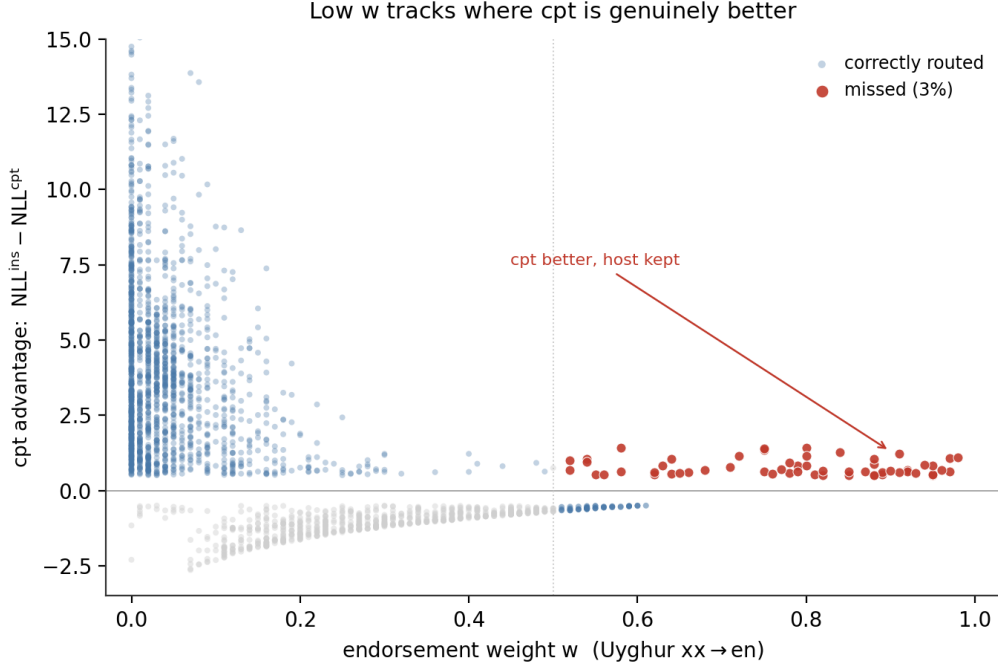


Figure 4. Each point is a token where the two models differ, for Uyghur into English. Low w falls where the CPT model is much better (upper left). High w falls where the host is better (lower band). The red points are the 3% where the CPT model is better but the host is kept. This residual is the headroom the oracle recovers.

The effect needs long outputs. The structure does not appear on short tasks. On reading comprehension, with two tokens per example, w behaves the same for strong and weak CPT models, with a mean of 0.589 for both and 78% agreement. With so few tokens, the gate has no room to act. The structure appears over the longer outputs of translation and title generation. This is also where SAF-W gains the most over fixed-weight fusion.

6.3 Headroom of Host Selection

Host selection operates at the task and language level. A per-sample oracle that selects the host for each test example marks the ceiling. The gap between SAF-W and this oracle is the headroom (Appendix C). It comes from the missed tokens above. A router that selects the host from the input is a natural way to close it.

6.4 Why Math Needs Uniform Averaging

Math is the one task where the endorsement offset hurts rather than helps, and where SAF-W instead decodes at its $\delta_t = 0$ reduction. The generated outputs show why. A math problem in this benchmark needs two abilities in sequence. The CPT model must first read the low-resource question, and the large model must then carry out the arithmetic. The endorsement offset biases each token toward a single model, and this cuts the two-step chain short. With the offset, SAF-W and the three-model TriMix both abandon the reasoning chain and collapse into bare arithmetic over the surface numbers, applying the wrong operation (Table 10). Fixing $\delta_t = 0$ keeps both models active at every token and restores the chain, so the model reads the problem before it

computes. Development data selects $\delta_t=0$ for math and the offset for every other task, with no test labels. The fix stays inside our framework, because uniform averaging is the $\delta_t=0$ case of SAF-W rather than a separate method.

Scale	Lang	Single		Fusion			SAF-W host	
		ins	cpt	Unif. avg	Proxy	TriMix	ins-host	cpt-host
7B	Tib	13.0	5.0	11.5	11.0	5.5	7.5	13.5
	Uig	30.0	27.0	43.5	36.5	35.0	34.0	33.0
	Mon	12.0	4.5	16.0	9.5	8.0	7.0	15.5
	Kaz	13.0	21.0	29.0	18.0	28.0	26.5	16.0
14B	Tib	5.5	5.0	22.0	12.0	7.0	10.0	8.0
	Uig	13.0	27.0	49.0	41.0	34.5	35.5	15.5
	Mon	5.0	4.5	11.0	6.0	7.0	4.5	6.5
	Kaz	12.0	21.0	44.5	19.0	25.0	24.0	18.5
32B	Tib	2.5	5.0	22.5	–	8.0	9.0	1.5
	Uig	4.5	27.0	45.0	–	33.0	30.5	8.0
	Mon	3.0	4.5	16.5	–	8.0	4.5	5.0
	Kaz	1.5	21.0	23.5	–	23.5	25.0	2.0

Table 9. Math accuracy (%) across all methods. Uniform averaging is the $\delta_t=0$ case of SAF-W. It outperforms every biased fusion method (Proxy, TriMix, and both SAF-W host assignments) at almost every scale and language, because biased fusion suppresses the reasoning chain (Table 10). This is why SAF-W decodes math at $\delta_t=0$. **Bold** marks uniform averaging.

Problem 1 [Uyghur]: *A volleyball costs 30 yuan and a basketball 40 yuan. What is the total cost of one volleyball and one basketball?* (gold: 70)

Uniform ($\delta_t=0$) ✓	avg.	1. The cost of one volleyball is 30 yuan, and the cost of one basketball is 40 yuan. The cost of one volleyball and one basketball is $30 + 40 = 70$ yuan.
SAF-W ✗		$100 - 30 - 40 - 50 = -70$. The answer is -70
TriMix ✗		$100 - 30 - 40 - 50 = 10$. The answer is 10

Problem 2 [Kazakh]: *A mango costs 25.60 yuan and a quince 18.80 yuan. What is their total cost?* (gold: 44.40)

Uniform ($\delta_t=0$) ✓	avg.	1. The cost of the mango is 25.60 yuan. 2. The cost of the quince is 18.80 yuan. 3. The total cost of the quince and the mango is $25.60 + 18.80 = 44.40$ yuan.
SAF-W ✗		$18.80 + 50 = 68.80$. $25.60 - 68.80 = -43.20$. The answer is -43.20
TriMix ✗		$25.60 + 50 = 75.60$. $75.60 - 18.80 = 56.80$. The answer is 56.80

Problem 3 [Mongolian]: *There are 522 items to be distributed at 87 per day. How many days are needed?* (gold: 6)

Uniform ($\delta_t=0$) ✓	avg.	$522 / 87 = 6$. Therefore, the number of days is 6. The answer is 6
SAF-W ✗		$87 \times 522 = 45314$. The answer is 45314
TriMix ✗		$522 \times 87 = 45614$. The answer is 45614

Problem 4 [Tibetan]: *Five years ago there were 64 sheep; the number has since increased by 28. How many are there now?* (gold: 92)

Uniform ($\delta_t=0$) ✓	avg.	5 years ago, the number of sheep was 64. The number of sheep increased by 28, so the current number of sheep is $64 + 28 = 92$. The answer is 92
SAF-W ✗		$5 \times 64 = 320$. $320 - 28 = 292$. $292 / 28 = 10$. $10 \times 28 = 280$. $320 - 280 = 40$. The answer is 40
TriMix ✗		$5 \times 64 = 320$. $320 - 28 = 292$. $292 / 28 = 10$. $10 \times 28 = 280$. The answer is 280

Table 10. Host-trusting fusion suppresses the reasoning chain on math. Uniform averaging ($\delta_t=0$) reads the problem before computing. Both SAF-W and TriMix collapse the chain into bare arithmetic over surface numbers, applying the wrong operation. Since uniform averaging is the $\delta_t=0$ case of SAF-W, the fix lies within our framework, and math alone is decoded at $\delta_t=0$.

7 Bonus: H3a, a Disagreement-Weighted Decoder at 70B

We also report a second two-model decoder, developed independently of SAF-W and evaluated on a different benchmark. We call it H3a. It answers the same question as SAF-W, namely how two frozen models should share one decoding step, but with a different signal and a different fusion rule. SAF-W reads a one-sided endorsement of the host’s proposed token and mixes logits convexly. H3a reads a symmetric disagreement between the two full distributions and pools them log-linearly.

Method. A planner A and a verifier B score each step t , giving next-token distributions p_t^A and p_t^B . H3a decodes in five steps.

Step 1, disagreement. The per-token signal is the normalized Jensen–Shannon distance between the two distributions. With $m_t = \frac{1}{2}(p_t^A + p_t^B)$,

$$\text{JSD}(p_t^A \parallel p_t^B) = \frac{1}{2} \text{KL}(p_t^A \parallel m_t) + \frac{1}{2} \text{KL}(p_t^B \parallel m_t), \quad (11)$$

$$\delta_t^{\text{raw}} = \sqrt{\text{JSD}(p_t^A \parallel p_t^B) / \ln 2} \in [0, 1]. \quad (12)$$

Step 2, smoothing. An exponential moving average stabilizes the signal over the sequence,

$$\delta_t = \beta \delta_{t-1} + (1 - \beta) \delta_t^{\text{raw}}, \quad \beta = 0.9, \quad (13)$$

with $\delta_1 = \delta_1^{\text{raw}}$ at the first step.

Step 3, dynamic gain. A gain λ turns disagreement into influence, scaled by the verifier’s confidence in its own top token,

$$\lambda_{\text{eff}} = \lambda \cdot \max_v p_t^B(v). \quad (14)$$

Step 4, weights. The verifier weight is $w_t = \lambda_{\text{eff}} \delta_t$, normalized to

$$\alpha_B = \frac{w_t}{1 + w_t}, \quad \alpha_A = \frac{1}{1 + w_t}. \quad (15)$$

Step 5, log-linear pooling. The fused distribution is $\log \pi_t(v) = \alpha_A \log p_t^A(v) + \alpha_B \log p_t^B(v)$, renormalized and decoded greedily. When the two models agree, δ_t is small and the planner decodes alone. Disagreement hands weight to the verifier.

Setup. We evaluate on the home benchmark of Proxy Tuning (Liu et al., 2024) at 70B. The planner is Llama-2-70B. The verifier is Llama-2-7B-chat for the instruction-following tasks and a 7B TriviaQA expert for the task-tuning test. Proxy Tuning runs three models totalling 84B. H3a runs two models totalling 77B. We use $\lambda = 2$ throughout. All reference numbers come from the original paper.

Results. Table 11 reports the comparison. H3a is competitive with Proxy Tuning across the evaluated tasks while using only two models. It improves TruthfulQA MC by 2.4 points, from 59.2

to 61.6, and TruthfulQA Open by 1.1 points, from 85.1 to 86.2. On TruthfulQA Open, H3a also exceeds the directly tuned 70B-chat model by 6.6 points. The two fusion methods are nearly tied on GSM, with scores of 31.7 and 32.0, while H3a is slightly worse on ToxiGen, where its toxicity rate is 0.29 compared with 0.0 for Proxy Tuning. On TriviaQA, H3a reaches 63.0, marginally exceeding Proxy Tuning at 62.7 and nearly matching the directly tuned 70B-chat model at 63.1. Overall, these results show that disagreement-weighted two-model decoding can match or slightly outperform three-model Proxy Tuning at 70B, although the gains remain task dependent.

Task	Verifier	H3a	Proxy Tuning	70B-chat	70B base
TruthfulQA MC	7B-chat	61.6	59.2	68.3	42.3
TruthfulQA Open	7B-chat	86.2	85.1	79.6	53.9
ToxiGen (↓)	7B-chat	0.29	0.0	0.0	67.4
GSM	7B-chat	31.7	32.0	51.8	9.6
TriviaQA	7B-TriviaQA-FT	63.0	62.7	63.1	45.2

Table 11. H3a against Proxy Tuning at 70B on the benchmarks of [Liu et al. \(2024\)](#), all with the dynamic gain. The H3a planner is Llama-2-70B in every row, paired with the listed verifier, so H3a runs two models. Proxy Tuning steers the same Llama-2-70B base with the same tuned 7B model as expert and Llama-2-7B as anti-expert, so it runs three models. 70B-chat is the directly tuned model, and 70B base is the untuned planner alone. All reference numbers come from the original paper. **Bold** marks the better fusion method.

8 Conclusion

This report studied decoding-time fusion for adapting large language models to low-resource languages. We reproduced two existing fusion methods, Proxy Tuning and TriMix, and evaluated them on MiLiC-Eval across four languages and three model scales. We identified a dilution effect, where a fixed mixture loses ground to the single large model as that model grows stronger, and at 32B drops below it. We then proposed SAF-W, a fusion method that trusts one host model by default and uses a tuning-free per-token endorsement from the second model to decide when to intervene. SAF-W uses two models against the three of Proxy Tuning and TriMix, and its advantage over TriMix grows with scale, from +2.0% at 7B to +9.4% at 32B. A token-level analysis showed that the endorsement acts only on the rare tokens where one model is clearly better, and that the host choice depends on which model is stronger on its own. This quantity needs no test labels. The devx choice matches the test-best host in 53 of 60 cells.

Host selection has three levels. The host can be fixed in three ways. The three levels bound each other. A single choice per task and language on the devx set is deployable and is what SAF-W uses. A per-task choice that always picks the test-best host marks an intermediate level. A per-example oracle that consults the test label marks the ceiling. Appendix C reports the three levels as the devx-select, max, and oracle rows. The gap between the devx choice and the oracle is the headroom that a learned router could recover. This per-example bound is tighter than the task-level upper bound reported for fixed-weight fusion, because it varies the host within a task rather than across tasks.

9 Limitations

We note several limitations of SAF-W and of the evaluation in this report. Each suggests a direction for future work.

Selection relies on a small development set. SAF-W fixes two discrete choices on held-out development data, the host model and whether to apply the endorsement offset. These sets are small, with 40 examples for reading comprehension and mathematical reasoning and 80 for the remaining tasks. A set of this size gives a noisy estimate of the development metric, so the configuration selected on development data may not be the one that performs best on the test distribution. The risk is largest on tasks where the two host assignments score close to each other, where a few examples can flip the choice. A larger development set, or a selection criterion that aggregates across related tasks, would make the deployed configuration more reliable. We report the development scores behind every choice in Appendix C so that the margin of each selection is visible.

Fusion assumes a shared tokenizer. SAF-W fuses logits at the token level and therefore assumes that the host and the scorer share a vocabulary. This holds when both models come from the same family, the setting we study, but not across families, where the two tokenizers segment text differently. Extending SAF-W across families would require aligning the two vocabularies before the per-token endorsement can be computed. That is a separate problem from the one addressed here. The same tokenization issue also affects the underlying adaptation. Continued pretraining gives smaller gains on scripts that the base tokenizer segments poorly, an effect observed for Mongolian in prior work (Zhang et al., 2026), and this limits how much language competence the scorer can carry for such scripts.

The endorsement is a one-sided signal. The endorsement reads only whether the scorer assigns high probability to the token the host proposed. It does not read whether the scorer would have proposed a better token of its own. When the scorer accepts the host token but privately prefers a different one, the weight stays high and the better token is missed. These cases are a small fraction of the tokens where the scorer is clearly more accurate, but they account for the gap between SAF-W and the per-example oracle. A two-sided signal that also reads the scorer’s own top token could recover this gap.

Gains depend on output length. The benefit of the endorsement appears only when outputs are long enough for the per-token weighting to act. It is largest on translation and title generation, the tasks with extended outputs, and absent on short classification tasks such as reading comprehension, where each example yields only a few tokens. SAF-W therefore offers little advantage over uniform averaging on short-output tasks, and its value is concentrated in generation.

Few scale pairs per family. SAF-W combines a small CPT scorer with a larger host, and the scaling trend we report rests on a small number of such pairs. For Qwen2.5 the CPT model is fixed at 1.5B and the host takes three sizes. For Gemma-3 the CPT model is fixed at 4B and the

host takes only two, 12B and 27B. For Llama the only pair is a 13B host with a 7B CPT model, in the code-switching test of §5.5, and Mixtral contributes a single 8x7B host there. We therefore observe the behavior of SAF-W at a handful of scale points within each family, rather than across a dense range of host and scorer sizes. The finding that the advantage grows with host size holds at the points we test, but a denser sweep, pairing CPT models of different sizes with hosts across more scales, would establish the trend more firmly and show how it depends on the gap between the two models.

References

- Yu Fei, Yasaman Razeghi, and Sameer Singh. Nudging: Inference-time alignment of LLMs via guided decoding. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12702–12739, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.623.
- Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025. URL <https://arxiv.org/abs/2503.19786>.
- Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.674.
- Ayoub Hammal, Pierre Zweigenbaum, and Caio Corro. KAD: A framework for proxy-based test-time alignment with knapsack approximation deferral. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3854–3872, Rabat, Morocco, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.eacl-long.179.
- Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002. doi: 10.1162/089976602760128018.
- Pratik Joshi, Sebastin Santy, Amar Budhiraja, Kalika Bali, and Monojit Choudhury. The state and fate of linguistic diversity and inclusion in the NLP world. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6282–6293. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.560. URL <https://aclanthology.org/2020.acl-main.560>.
- Yeeun Kang. CoVoSwitch: Machine translation of synthetic code-switched text based on intonation units. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 4: Student Research Workshop)*, pages 469–481, Bangkok, Thailand, 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.acl-srw.40>.
- Duong Minh Le, Yang Chen, Alan Ritter, and Wei Xu. Constrained decoding for cross-lingual label projection. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=DayPQXaQk>.

- Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, 2004. Association for Computational Linguistics. URL <https://aclanthology.org/W04-1013/>.
- Alisa Liu, Maarten Sap, Ximing Lu, Swabha Swayamdipta, Chandra Bhagavatula, Noah A. Smith, and Yejin Choi. DExperts: Decoding-time controlled text generation with experts and anti-experts. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6691–6706, Online, 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.522. URL <https://aclanthology.org/2021.acl-long.522>.
- Alisa Liu, Xiaochuang Han, Yizhong Wang, Yulia Tsvetkov, Yejin Choi, and Noah A. Smith. Tuning language models by proxy. In *First Conference on Language Modeling (COLM)*, 2024. URL <https://arxiv.org/abs/2401.08565>.
- Arijit Nag, Soumen Chakrabarti, Animesh Mukherjee, and Niloy Ganguly. Efficient continual pre-training of LLMs for low-resource languages. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*, pages 304–317, Albuquerque, New Mexico, 2025. Association for Computational Linguistics. ISBN 979-8-89176-194-0.
- Maja Popović. chrF++: words helping character n-grams. In *Proceedings of the Second Conference on Machine Translation*, pages 612–618, Copenhagen, Denmark, 2017. Association for Computational Linguistics. doi: 10.18653/v1/W17-4770. URL <https://aclanthology.org/W17-4770/>.
- Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL <https://arxiv.org/abs/2412.15115>.
- Shivalika Singh, Angelika Romanou, Clémentine Fourier, David Ifeoluwa Adelani, Jian Gang Ngui, Daniel Vila-Suero, Peerat Limkonchotiwat, Kelly Marchisio, Wei Qi Leong, Yosephine Susanto, Raymond Ng, Shayne Longpre, Sebastian Ruder, Wei-Yin Ko, Antoine Bosselut, Alice Oh, Andre Martins, Leshem Choshen, Daphne Ippolito, et al. Global MMLU: Understanding and addressing cultural and linguistic biases in multilingual evaluation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18761–18799, Vienna, Austria, 2025. Association for Computational Linguistics.
- Mingxu Tao, Chen Zhang, Quzhe Huang, Tianyao Ma, Songfang Huang, Dongyan Zhao, and Yansong Feng. Unlocking the potential of model merging for low-resource languages. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 8705–8720, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.508.
- Gorka Urbizu, Ander Corral, Xabier Saralegi, and Iñaki San Vicente. Sub-1B language models for low-resource languages: Training strategies and insights for Basque. In *Proceedings of the 5th Workshop on Multilingual Representation Learning (MRL 2025)*. Association for Computational Linguistics, 2025.

- Chen Zhang, Mingxu Tao, Zhiyuan Liao, and Yansong Feng. MiLiC-Eval: Benchmarking multilingual LLMs for China’s minority languages. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 11086–11102, Vienna, Austria, 2025. Association for Computational Linguistics. URL <https://aclanthology.org/2025.findings-acl.578/>.
- Chen Zhang, Jiuheng Lin, Zhiyuan Liao, and Yansong Feng. Efficient low-resource language adaptation via multi-source dynamic logit fusion. *arXiv preprint arXiv:2604.18106*, 2026. URL <https://arxiv.org/abs/2604.18106>.
- Xiang Zhang, Senyu Li, Bradley Hauer, Ning Shi, and Grzegorz Kondrak. Don’t trust ChatGPT when your question is not in English: A study of multilingual abilities and types of LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7915–7927, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.491.

A README.md File

A copy of the README.md file is attached to this document and can be found on the final pages, after the per-language results.

B Use of Auto-generation Tools

B.1 Claude

Claude (Anthropic) assisted the author in three areas. It assisted in writing this report. The author supplied the results, the claims, the structure, and the terminology, and Claude drafted and revised LaTeX passages, tables, and captions throughout the report to the author's specification. It assisted in the experiment code, including a memory optimisation of the decoding loop, per-sample evaluation scripts, and cluster job commands. It assisted in aggregating metric outputs into the result tables and in recomputing row averages. The author reviewed every passage and verified every reported number against the underlying metric files before inclusion.

B.2 GitHub Copilot

GitHub Copilot assisted the author in producing function docstrings and short descriptive comments in the project repository. The author gave no specific commands. Copilot suggestions appeared as inline completions during editing and were occasionally accepted. The author reviewed every accepted suggestion.

C Full Per-Language Results

Section C.1 covers Qwen2.5 and Section C.2 covers Gemma-3, one table per language, all six tasks, every scale. Uniform averaging is the $\delta_t=0$ ablation of SAF-W. Proxy Tuning uses weights 1.0/1.0. Ins-host and cpt-host are the two host assignments. Devx-select is the deployed configuration, with the host from Table 7 and uniform averaging on math. Max[†] keeps the test-best host per cell and oracle[†] per example. Both use test references. Missing cells are marked –, and their row averages cover available cells only. Average excludes math. Bold marks cells $\geq$ TriMix.

C.1 Qwen2.5

Model	Read. Comp.	Resp. Sel.	Title Gen.	MT xx2en	MT en2xx	Average	Math	Avg. w/ Math
Qwen2.5-1.5B-base	43.8	35.1	21.6	8.4	8.8	23.5	9.3	21.2
Qwen2.5-1.5B-cpt (ours)	43.5	32.4	28.0	7.1	16.3	25.5	5.0	22.1
Qwen2.5-1.5B-ins	34.3	33.3	13.5	10.6	11.9	20.7	6.5	18.3
Qwen2.5-7B-ins	47.0	37.1	29.3	13.0	15.4	28.4	13.0	25.8
Uniform averaging (cpt+7B)	45.5	37.1	32.2	9.5	15.3	27.9	11.5	25.2
Proxy Tuning (cpt+7B)	46.5	38.8	28.3	14.6	17.7	29.2	11.0	26.2
TriMix (cpt+7B)	46.5	38.1	29.1	6.8	15.7	27.2	5.5	23.6
SAF-W ins-host (cpt+7B)	49.0	38.3	29.3	13.3	15.8	29.1	7.5	25.5
SAF-W cpt-host (cpt+7B)	46.5	37.6	28.8	13.3	14.6	28.2	13.5	25.7
SAF-W devx-select (cpt+7B)	46.5	38.3	28.8	13.3	15.8	28.5	11.5	25.7
max [†] (cpt+7B)	49.0	38.3	29.3	13.3	15.8	29.1	13.5	26.5
oracle [†] (cpt+7B)	52.5	45.2	35.6	15.8	17.4	33.3	17.0	30.6
Qwen2.5-14B-ins	53.5	40.0	32.9	21.4	19.4	33.4	5.5	28.8
Proxy Tuning (cpt+14B)	53.0	39.8	26.0	21.9	20.1	32.2	12.0	28.8
TriMix (cpt+14B)	51.0	39.6	29.8	7.6	16.2	28.8	7.0	25.2
SAF-W ins-host (cpt+14B)	52.5	39.6	29.6	15.8	15.9	30.7	10.0	27.2
SAF-W cpt-host (cpt+14B)	53.0	39.6	32.1	21.6	18.9	33.0	8.0	28.9
SAF-W devx-select (cpt+14B)	53.0	39.6	32.1	21.6	18.9	33.0	–	–
max [†] (cpt+14B)	53.0	39.6	32.1	21.6	18.9	33.0	10.0	29.2
oracle [†] (cpt+14B)	58.0	46.2	37.5	22.9	20.2	37.0	16.0	33.5
Qwen2.5-32B-ins	57.5	42.3	31.5	21.3	19.2	34.4	2.5	29.1
TriMix (cpt+32B)	49.5	37.8	30.5	7.9	16.2	28.4	8.0	25.0
SAF-W ins-host (cpt+32B)	52.0	41.5	32.4	16.4	16.1	31.7	9.0	27.9
SAF-W cpt-host (cpt+32B)	58.5	40.8	33.5	21.1	18.6	34.5	1.5	29.0
SAF-W devx-select (cpt+32B)	58.5	41.5	33.5	21.1	18.6	34.6	–	–
max [†] (cpt+32B)	58.5	41.5	33.5	21.1	18.6	34.6	9.0	30.4
oracle [†] (cpt+32B)	61.5	47.2	38.6	22.5	19.9	37.9	9.0	33.1

Table 12. Scores (%) on **Tibetan** tasks of MiLiC-Eval (Qwen). Rows are defined at the start of this appendix. **Bold** marks cells $\geq$ TriMix.

Model	Read. Comp.	Resp. Sel.	Title Gen.	MT xx2en	MT en2xx	Average	Math	Avg. w/ Math
Qwen2.5-1.5B-base	48.7	38.7	15.3	13.1	6.6	24.5	16.2	23.1
Qwen2.5-1.5B-cpt (ours)	65.0	45.7	16.7	36.5	20.0	36.8	27.0	35.1
Qwen2.5-1.5B-ins	44.2	36.4	14.4	14.1	9.4	23.7	12.3	21.8
Qwen2.5-7B-ins	57.0	47.9	19.9	26.0	11.8	32.5	30.0	32.1
Uniform averaging (cpt+7B)	57.5	49.1	20.8	32.7	14.8	34.9	43.5	36.4
Proxy Tuning (cpt+7B)	57.0	48.2	16.4	31.9	12.8	33.3	36.5	33.8
TriMix (cpt+7B)	64.0	51.6	16.5	37.9	20.2	38.0	35.0	37.5
SAF-W ins-host (cpt+7B)	62.5	51.3	16.9	37.7	19.2	37.5	34.0	36.9
SAF-W cpt-host (cpt+7B)	58.0	47.9	20.0	27.4	11.8	33.0	33.0	33.0
SAF-W devx-select (cpt+7B)	62.5	51.3	20.0	37.7	19.2	38.1	43.5	39.0
max [†] (cpt+7B)	62.5	51.3	20.0	37.7	19.2	38.1	34.0	37.4
oracle [†] (cpt+7B)	64.5	56.0	23.3	39.4	20.0	40.6	47.0	41.7
Qwen2.5-14B-ins	63.5	58.7	23.8	30.7	14.4	38.2	13.0	34.0
Proxy Tuning (cpt+14B)	63.5	57.7	20.5	36.1	16.0	38.8	41.0	39.1
TriMix (cpt+14B)	66.0	57.7	18.1	38.2	22.5	40.5	34.5	39.5
SAF-W ins-host (cpt+14B)	69.5	59.7	18.5	38.1	20.7	41.3	35.5	40.3
SAF-W cpt-host (cpt+14B)	66.0	58.7	23.6	32.2	15.2	39.1	15.5	35.2
SAF-W devx-select (cpt+14B)	69.5	58.7	23.6	38.1	20.7	42.1	–	–
max [†] (cpt+14B)	69.5	59.7	23.6	38.1	20.7	42.3	35.5	41.2
oracle [†] (cpt+14B)	71.5	65.1	27.6	40.7	22.5	45.5	46.0	45.6
Qwen2.5-32B-ins	65.5	65.8	26.4	33.3	16.4	41.5	4.5	35.3
TriMix (cpt+32B)	71.5	65.1	17.1	38.1	22.5	42.9	33.0	41.2
SAF-W ins-host (cpt+32B)	73.0	67.6	17.9	38.4	21.1	43.6	30.5	41.4
SAF-W cpt-host (cpt+32B)	66.0	66.6	24.6	34.7	17.7	41.9	8.0	36.3
SAF-W devx-select (cpt+32B)	73.0	67.6	24.6	38.4	21.1	44.9	–	–
max [†] (cpt+32B)	73.0	67.6	24.6	38.4	21.1	44.9	30.5	42.5
oracle [†] (cpt+32B)	74.0	71.0	27.6	42.2	23.8	47.7	36.0	45.8

Table 13. Scores (%) on **Uyghur** tasks of MiLiC-Eval (Qwen). Rows are defined at the start of this appendix. **Bold** marks cells $\geq$ TriMix.

Model	Read. Comp.	Resp. Sel.	Title Gen.	MT xx2en	MT en2xx	Average	Math	Avg. w/ Math
Qwen2.5-1.5B-base	38.5	32.7	10.1	10.1	7.0	19.7	9.3	17.9
Qwen2.5-1.5B-cpt (ours)	44.5	28.7	14.7	6.3	12.6	21.4	4.5	18.6
Qwen2.5-1.5B-ins	32.7	31.5	3.9	12.6	7.3	17.6	7.3	15.9
Qwen2.5-7B-ins	44.0	39.3	11.6	13.2	4.0	22.4	12.0	20.7
Uniform averaging (cpt+7B)	44.0	40.5	15.0	9.8	8.1	23.5	16.0	22.2
Proxy Tuning (cpt+7B)	43.0	39.8	9.1	12.7	0.9	21.1	9.5	19.2
TriMix (cpt+7B)	43.0	40.5	15.4	6.0	11.3	23.2	8.0	20.7
SAF-W ins-host (cpt+7B)	44.0	41.5	15.2	11.9	11.5	24.8	7.0	21.8
SAF-W cpt-host (cpt+7B)	44.0	41.3	13.6	13.9	6.0	23.8	15.5	22.4
SAF-W devx-select (cpt+7B)	44.0	41.3	15.2	13.9	11.5	25.2	16.0	23.6
max [†] (cpt+7B)	44.0	41.5	15.2	13.9	11.5	25.2	15.5	23.6
oracle [†] (cpt+7B)	47.5	46.4	18.3	15.2	11.9	27.9	20.0	26.6
Qwen2.5-14B-ins	47.5	46.7	13.6	16.1	0.7	24.9	5.0	21.6
Proxy Tuning (cpt+14B)	46.0	45.2	10.5	16.5	8.8	25.4	6.0	22.2
TriMix (cpt+14B)	49.0	38.8	15.6	6.0	12.3	24.3	7.0	21.4
SAF-W ins-host (cpt+14B)	46.0	41.5	14.9	13.2	11.3	25.4	4.5	21.9
SAF-W cpt-host (cpt+14B)	47.5	45.0	14.8	15.8	6.1	25.8	6.5	22.6
SAF-W devx-select (cpt+14B)	47.5	41.5	14.9	15.8	11.3	26.2	–	–
max [†] (cpt+14B)	47.5	45.0	14.9	15.8	11.3	26.9	6.5	23.5
oracle [†] (cpt+14B)	51.0	48.4	18.9	17.0	11.8	29.4	10.0	26.2
Qwen2.5-32B-ins	49.5	45.9	15.9	15.8	7.9	27.0	3.0	23.0
TriMix (cpt+32B)	47.0	42.0	15.8	6.0	12.3	24.6	8.0	21.8
SAF-W ins-host (cpt+32B)	47.0	43.7	15.5	12.4	12.4	26.2	4.5	22.6
SAF-W cpt-host (cpt+32B)	49.5	46.0	16.1	15.4	7.6	26.9	5.0	23.3
SAF-W devx-select (cpt+32B)	49.5	46.0	16.1	15.4	12.4	27.9	–	–
max [†] (cpt+32B)	49.5	46.0	16.1	15.4	12.4	27.9	5.0	24.1
oracle [†] (cpt+32B)	52.0	48.9	18.8	16.5	13.0	29.8	9.0	26.4

Table 14. Scores (%) on **Mongolian** tasks of MiLiC-Eval (Qwen). Rows are defined at the start of this appendix. **Bold** marks cells $\geq$ TriMix.

Model	Read. Comp.	Resp. Sel.	Title Gen.	MT xx2en	MT en2xx	Average	Math	Avg. w/ Math
Qwen2.5-1.5B-base	46.5	35.6	16.9	11.0	5.8	23.2	8.7	20.8
Qwen2.5-1.5B-cpt (ours)	55.0	43.5	22.5	25.5	12.9	31.9	21.0	30.1
Qwen2.5-1.5B-ins	38.8	34.0	14.7	13.6	8.8	22.0	6.2	19.3
Qwen2.5-7B-ins	52.5	40.3	17.1	20.6	0.4	26.2	13.0	24.0
Uniform averaging (cpt+7B)	55.5	40.5	19.3	25.1	6.6	29.4	29.0	29.3
Proxy Tuning (cpt+7B)	55.0	39.3	18.5	24.2	1.4	27.7	18.0	26.1
TriMix (cpt+7B)	58.5	46.2	23.7	28.6	12.7	33.9	28.0	32.9
SAF-W ins-host (cpt+7B)	60.5	43.5	23.8	30.2	12.3	34.1	26.5	32.8
SAF-W cpt-host (cpt+7B)	54.0	39.8	18.9	21.5	1.1	27.1	16.0	25.2
SAF-W devx-select (cpt+7B)	54.0	43.5	23.8	30.2	12.3	32.8	29.0	32.1
max [†] (cpt+7B)	60.5	43.5	23.8	30.2	12.3	34.1	26.5	32.8
oracle [†] (cpt+7B)	63.5	45.7	27.6	31.7	12.3	36.2	32.0	35.5
Qwen2.5-14B-ins	58.0	47.4	16.4	24.8	6.0	30.5	12.0	27.4
Proxy Tuning (cpt+14B)	60.5	49.1	19.9	29.2	10.6	33.9	19.0	31.4
TriMix (cpt+14B)	59.5	53.1	22.6	29.6	13.5	35.7	25.0	33.9
SAF-W ins-host (cpt+14B)	61.0	52.1	24.9	31.4	12.3	36.3	24.0	34.3
SAF-W cpt-host (cpt+14B)	58.5	47.7	19.7	25.7	9.7	32.3	18.5	30.0
SAF-W devx-select (cpt+14B)	61.0	52.1	24.9	31.4	12.3	36.3	–	–
max [†] (cpt+14B)	61.0	52.1	24.9	31.4	12.3	36.3	24.0	34.3
oracle [†] (cpt+14B)	64.5	56.0	29.3	33.9	13.9	39.5	36.5	39.0
Qwen2.5-32B-ins	58.5	53.3	21.9	27.4	11.8	34.6	1.5	29.1
TriMix (cpt+32B)	61.0	55.3	22.6	28.7	14.0	36.2	23.5	34.3
SAF-W ins-host (cpt+32B)	61.0	55.3	24.1	32.6	13.2	37.2	25.0	35.2
SAF-W cpt-host (cpt+32B)	59.0	52.8	24.2	28.6	12.8	35.5	2.0	29.9
SAF-W devx-select (cpt+32B)	59.0	55.3	24.2	32.6	12.8	36.8	–	–
max [†] (cpt+32B)	61.0	55.3	24.2	32.6	13.2	37.3	25.0	35.2
oracle [†] (cpt+32B)	63.0	57.5	31.1	36.1	16.0	40.7	26.0	38.3

Table 15. Scores (%) on **Kazakh** tasks of MiLiC-Eval (Qwen). Rows are defined at the start of this appendix. **Bold** marks cells $\geq$ TriMix.

C.2 Gemma-3

Model	Read. Comp.	Resp. Sel.	Title Gen.	MT xx2en	MT en2xx	Average	Math	Avg. w/ Math
gemma-3-4b-bo-cpt (ours)	62.0	55.8	5.4	46.7	39.2	41.8	34.5	40.6
gemma-3-12B-it	62.0	55.8	5.4	39.5	24.0	37.3	34.5	36.9
TriMix (cpt+12B)	69.5	64.4	48.3	39.8	38.6	52.1	45.0	50.9
SAF-W devx-select (cpt+12B)	71.5	61.4	48.4	46.1	37.7	53.0	58.5	53.9
oracle [†] (cpt+12B)	73.5	63.1	51.4	47.7	38.2	54.8	66.5	56.7
gemma-3-27B-it	62.0	67.3	4.3	39.7	25.6	39.8	31.5	38.4
TriMix (cpt+27B)	68.5	69.5	48.3	38.4	38.8	52.7	45.0	51.4
SAF-W devx-select (cpt+27B)	68.0	68.3	48.0	47.7	38.0	54.0	48.5	53.1
oracle [†] (cpt+27B)	70.5	71.2	51.1	49.6	38.5	56.2	59.0	56.6

Table 16. Scores (%) on **Tibetan** tasks of MiLiC-Eval (Gemma). The CPT model is gemma-3-4b-bo-cpt. Rows are defined at the start of this appendix. **Bold** marks cells $\geq$ TriMix.

Model	Read. Comp.	Resp. Sel.	Title Gen.	MT xx2en	MT en2xx	Average	Math	Avg. w/ Math
gemma-3-4b-ug-cpt (ours)	70.5	56.8	9.8	47.0	32.0	43.2	42.5	43.1
gemma-3-12B-it	77.0	81.6	9.8	43.8	20.3	46.5	71.5	50.7
TriMix (cpt+12B)	73.5	80.6	26.4	42.9	32.7	51.2	54.0	51.7
SAF-W devx-select (cpt+12B)	77.0	81.6	29.2	48.1	31.9	53.6	70.0	56.3
oracle [†] (cpt+12B)	82.5	84.0	34.8	50.3	33.3	57.0	74.0	59.8
gemma-3-27B-it	81.0	85.0	12.2	46.4	25.2	50.0	41.0	48.5
TriMix (cpt+27B)	78.5	82.6	26.7	44.4	32.8	53.0	53.0	53.0
SAF-W devx-select (cpt+27B)	81.5	85.3	28.7	49.7	32.6	55.6	53.5	55.2
oracle [†] (cpt+27B)	85.5	86.7	32.2	52.6	34.8	58.4	62.5	59.1

Table 17. Scores (%) on **Uyghur** tasks of MiLiC-Eval (Gemma). The CPT model is gemma-3-4b-ug-cpt. Rows are defined at the start of this appendix. **Bold** marks cells $\geq$ TriMix.

Model	Read. Comp.	Resp. Sel.	Title Gen.	MT xx2en	MT en2xx	Average	Math	Avg. w/ Math
gemma-3-4b-mn-cpt (ours)	49.5	34.2	1.0	19.4	14.2	23.7	26.5	24.1
gemma-3-12B-it	43.5	41.3	4.6	14.5	11.8	23.1	12.5	21.4
TriMix (cpt+12B)	47.5	41.5	15.1	10.6	13.8	25.7	25.0	25.6
SAF-W devx-select (cpt+12B)	47.0	43.2	16.4	18.0	13.9	27.7	25.5	27.3
oracle [†] (cpt+12B)	50.0	50.4	21.8	20.0	15.1	31.5	31.0	31.4
gemma-3-27B-it	48.0	40.0	4.0	15.5	15.2	24.5	14.0	22.8
TriMix (cpt+27B)	49.0	41.3	15.9	9.6	14.4	26.0	–	–
SAF-W devx-select (cpt+27B)	48.5	44.0	16.9	18.8	14.6	28.6	25.5	28.1
oracle [†] (cpt+27B)	53.5	51.8	21.1	20.8	17.2	32.9	30.0	32.4

Table 18. Scores (%) on **Mongolian** tasks of MiLiC-Eval (Gemma). The CPT model is gemma-3-4b-mn-cpt. Rows are defined at the start of this appendix. The TriMix math cell at 27B is missing, so its six-task averages are marked –. **Bold** marks cells $\geq$ TriMix.

Model	Read. Comp.	Resp. Sel.	Title Gen.	MT xx2en	MT en2xx	Average	Math	Avg. w/ Math
gemma-3-4b-kk-cpt (ours)	66.0	61.9	9.3	43.4	29.1	41.9	37.0	41.1
gemma-3-12B-it	70.5	73.2	8.5	40.0	11.1	40.7	61.5	44.1
TriMix (cpt+12B)	71.5	74.0	29.0	40.6	28.6	48.7	44.5	48.0
SAF-W devx-select (cpt+12B)	74.5	73.5	28.0	44.3	28.2	49.7	63.5	52.0
oracle [†] (cpt+12B)	78.5	75.2	33.3	46.4	28.8	52.4	69.0	55.2
gemma-3-27B-it	76.5	79.4	7.5	45.4	19.1	45.6	28.5	42.7
TriMix (cpt+27B)	76.5	78.6	27.7	41.7	29.0	50.7	46.0	49.9
SAF-W devx-select (cpt+27B)	77.0	80.6	27.8	45.6	28.1	51.8	48.0	51.2
oracle [†] (cpt+27B)	79.5	83.3	33.6	48.5	30.3	55.0	55.0	55.0

Table 19. Scores (%) on **Kazakh** tasks of MiLiC-Eval (Gemma). The CPT model is gemma-3-4b-kk-cpt. Rows are defined at the start of this appendix. **Bold** marks cells $\geq$ TriMix.

Attachment: README.md

Constrained Decoding for Large Language Models

Test-Time Logit Fusion for Low-Resource Language Adaptation

This repository is the data analysis pipeline for a research project of the MPhil in Data Intensive Science at the University of Cambridge. The compiled project report and executive summary are in `report/`. The report describes the method and the results. This README describes how to install the pipeline, how to run every method, and how to reproduce every number in the report.

The project studies decoding-time logit fusion for adapting large language models to low-resource languages. Its objectives are to (i) reproduce two published fusion baselines, Proxy Tuning and TriMix, inside one controlled framework, (ii) propose SAF-W, a training-free per-token fusion of a host and a scorer model, (iii) select the SAF-W host per task and language on development data only, (iv) measure all methods across three backbone scales, four languages, and six tasks, and (v) confirm the findings on a second model family.

Data Availability

Datasets

The MiLiC-Eval splits ship with the repository under `data/`. The languages are Tibetan (bo), Uyghur (ug), Mongolian (mn), and Kazakh (kk). Each task directory holds one directory per language:

Task	Directory	Exemplars	Metric
Reading comprehension	<code>reading_comprehension</code>	5	accuracy
Response selection	<code>response_selection</code>	5	accuracy
Title generation	<code>title_generation_200</code>	3	ROUGE-L
Math (English CoT)	<code>math</code>	5	accuracy
Translation, both directions	<code>translation_dialogue</code>	5	chrF++

Each language directory holds a `test.json`, a small `dev.json`, and the exemplar files `train_1.json`, `train_2.json`, and `train_3.json`. Every example carries an `id` and the task fields the prompt builders in `safw/prompts.py` expect. All runs draw their few-shot exemplars from `train_1.json`, in file order.

Host selection uses an expanded development set called `devx`. The `devx` set merges `dev` with `train_2` and `train_3` and leaves `train_1` untouched, so the exemplar file never overlaps the selection data. The set holds 40 items for reading comprehension and 80 for the other selection tasks. `scripts/make_devx.py` builds it and de-duplicates by `id`.

Models

All checkpoints are frozen and public on Hugging Face:

Family	Role	Checkpoint
Qwen	host / base	Qwen/Qwen2.5-7B-Instruct, -14B-, -32B-
Qwen	scorer / expert	pkupie/Qwen2.5-1.5B-<lang>-cpt
Qwen	antiexpert (baselines)	Qwen/Qwen2.5-1.5B
Gemma	host / base	google/gemma-3-12b-it, -27b-it

Models load in bfloat16. On a cluster without internet access, set `HF_HUB_OFFLINE=1` and `TRANSFORMERS_OFFLINE=1` and point the Hugging Face cache at a pre-downloaded directory. The cluster wrapper `scripts/run.sh` sets both variables.

Results

Results live under `results/` in two trees. `results/devx/` holds the host-selection scores behind the selection table in the report. One file per task, language, and scale records both host scores on the devx set. `results/test_outputs/` holds every test run. `scripts/canonical.py` is the single source of truth for the layout:

```
results/test_outputs/{method}/{family}/{lang}/{task}/
  {method}_{family}_{lang}_{task}_{key}_preds.json
  {method}_{family}_{lang}_{task}_{key}_metrics.json
```

The key is `ins{scale}` or `cpt{size}` for single models, `{scale}` for the three-model baselines and uniform averaging, and `inshost{scale}` or `cpthost{scale}` for SAF-W. The predictions file maps example id to the generated text. The metrics file carries a metadata header, for example:

```
{
  "metric": "accuracy",
  "value": 0.585,
  "n": 200,
  "source_run": "saf_qwen_bo_rc_cpthost32B_preds.json",
  "key": "cpthost32B",
  "task": "rc",
  "lang": "bo",
  "family": "qwen",
  "method": "saf"
}
```

Files are populated from the CSD3 runs. They start as zero-byte placeholders. A zero-byte file never blocks a run and never counts as evaluated. Audit the unfilled files at any time with:

```
find results/test_outputs -name "*.json" -size 0 | wc -l
```

Installation

Requirements

Python 3.10 or newer. PyTorch 2.5 with CUDA, Transformers 4.47, Accelerate, sacrebleu, and SentencePiece, all pinned in `requirements.txt`. A CUDA GPU for inference. The 7B and 14B

pairs fit one 48 GB card. The 32B pairs need two 48 GB cards or one 80 GB card. Docker, if you prefer the containerised setup.

Local setup

Clone the repository, enter it, and install the package with its pinned dependencies:

```
git clone https://github.com/tianshuai-gao/SAF.git
cd SAF
pip install -e .
```

Install the multilingual ROUGE fork. Title generation is scored with the XL-Sum fork. It installs as `rouge_score` and must override the official package:

```
pip install pyonmttok
pip install "git+https://github.com/csebuatnlp/xl-sum.git#subdirectory=
    multilingual_rouge_scoring"
```

Verify the correct scorer is active:

```
python -c "from rouge_score import rouge_scorer; import inspect; \
assert 'kwargs' in str(inspect.signature(rouge_scorer.RougeScorer.__init__)), '
    wrong rouge'; \
print('multilingual fork OK')"
```

Do not install the official `rouge-score` package at any point. It shares the import name and silently deflates the title scores by roughly a factor of three. Optional extras install with `pip install -e ".[test]"` for pytest and `".[docs]"` for Sphinx.

Docker

Build the image from the repository root, run the unit tests inside it, then run inference with GPU access. Mount the working directory and a Hugging Face cache so weights are not baked into the image:

```
docker build -t safw:latest .
docker run --rm safw:latest pytest tests/ -q
docker run --gpus all -v $PWD:/workspace \
    -v /path/to/hf_cache:/root/.cache/huggingface \
    safw:latest python -m scripts.run --method safw --lang bo --scale 7B --host
    ins
```

The build installs everything above and runs the ROUGE assertion. A wrong scorer fails the build. On an HPC system without Docker, the same image runs through Apptainer with GPU pass-through:

```
apptainer exec --nv docker://tianshuai2024/safw:latest python -c \
    "import torch; assert torch.cuda.is_available()"
```

Determinism

Greedy decoding is used throughout, so there is no sampling. For bit-reproducible matrix operations on Ampere GPUs, set `CUBLAS_WORKSPACE_CONFIG=:4096:8`. The Docker image and the cluster wrapper set this automatically. The few-shot exemplars are fixed files read in order, so prompts are identical across runs.

Usage

Pipeline architecture

Three layers with one job each. `scripts/run.py` declares a batch of runs and validates the declaration. `scripts/infer.py` executes one run. `scripts/canonical.py` decides where every output lands. `scripts/eval.sh` sweeps the results tree and scores whatever is new. The task settings, namely the data paths, the exemplar counts, and the generation lengths, live once inside the driver.

The declaration driver

```
python -m scripts.run --method safw --lang bo --scale 32B --host cpt --dry_run
```

Flag	Meaning
-method	single, safw, safw_fixed, proxy, or trimix
-family	qwen (default) or gemma
-lang	bo, ug, mn, or kk
-scale	7B–32B, 12B, 27B. Not used by the single cpt model
-host	ins or cpt. SAF-W and single only
-tasks	any of rc rs title math xx2en en2xx. Defaults to all six
-host_model, -scorer_model	explicit SAF-W overrides
-base/expert/antiexpert_model	explicit baseline overrides
-beta_fixed	constant scorer weight for safw_fixed (default 0.5)
-alpha	Proxy Tuning residual weight (default 1.0)
-base_weight, -expert_weight	TriMix weights (defaults 1.0)
-plausibility_alpha	TriMix masking threshold (default 0.1)
-dry_run	print the plan and exit

Model paths resolve from a built-in zoo, so the common runs need no paths at all. Every declaration is cross-checked against the resolved model names. The driver refuses to run when the scale does not match the model, when the language does not match the CPT checkpoint, when a model sits in the wrong slot, when a flag does not apply to the method, or when a required declaration is missing. Each refusal exits non-zero with a one-line reason. `-dry_run` prints one plan line per task, including the exact output path, and exits.

Scripts

`scripts/infer.py` is the parametrised engine underneath the driver. It runs one task for one method with explicit paths:

```
python -m scripts.infer --method safw \  
  --host_model Qwen/Qwen2.5-7B-Instruct \  
  --scorer_model pkupie/Qwen2.5-1.5B-bo-cpt \  
  --task reading_comprehension --eval_lang bo --prompt_lang en \  
  --num_exemplar 5 --max_new_tokens 2 \  
  --input_file data/reading_comprehension/bo/test.json \  
  --exemplar_file data/reading_comprehension/bo/train_1.json \  
  --output_file /tmp/bo_rc_preds.json
```

`--method single` decodes `--host_model` on its own. `safw` and `safw_fixed` take the host and scorer models. `proxy` and `trimix` take the base, expert, and antiexpert models plus their weights. Task flags cover the exemplar count, the generation length, the title passage length, the translation directions, the batch size, and 8-bit loading. A non-empty output file stops the run, so nothing is ever overwritten.

`scripts/canonical.py` resolves the storage path for one run and can be called on its own. Pass the host or base model first. `scripts/evaluate.py` scores one predictions file against its reference file and writes the metrics file with the metadata header. `scripts/eval.sh` walks `results/test_outputs`, finds every non-empty predictions file without a metrics file, reads the task and the language from the path, and calls the evaluator. The sweep is idempotent. `scripts/get_perplexity.py` selects the TriMix weights by sweeping the grid `{0.1, 0.3, 0.5, 0.7, 0.9, 1.0}` and reporting the lowest-perplexity combination on a sampled development set, with no labels. `scripts/make_devx.py` builds the devx selection set by merging the dev split with the second and third exemplar splits and de-duplicating by id.

Reproduce the results from scratch

Decode the single-model rows, SAF-W under both hosts, uniform averaging, and Proxy Tuning:

```
for LANG in bo ug mn kk; do  
  python -m scripts.run --method single --lang $LANG --host cpt  
  for SCALE in 7B 14B 32B; do  
    python -m scripts.run --method single --lang $LANG --scale $SCALE --host ins  
    python -m scripts.run --method safw --lang $LANG --scale $SCALE --host ins  
    python -m scripts.run --method safw --lang $LANG --scale $SCALE --host cpt  
    python -m scripts.run --method safw_fixed --lang $LANG --scale $SCALE  
    python -m scripts.run --method proxy --lang $LANG --scale $SCALE  
  done  
done
```

The deployed SAF-W host per cell follows the devx selection recorded under `results/devx` and reported in the paper. For TriMix, first select the weights per task and language, then run with the selected values:

```
python -m scripts.get_perplexity --task reading_comprehension --eval_lang bo \  

```



```
--base_model Qwen/Qwen2.5-7B-Instruct \
--expert_model pkupie/Qwen2.5-1.5B-bo-cpt \
--antiexpert_model Qwen/Qwen2.5-1.5B
python -m scripts.run --method trimix --lang bo --scale 7B \
--tasks rc --base_weight 0.1 --expert_weight 1.0
```

Score everything and audit:

```
bash scripts/eval.sh
find results/test_outputs -name "*.json" -size 0 | wc -l
```

On the cluster, `scripts/run.sh` is a thin SBATCH wrapper around the driver for the CSD3 Ampere partition. It loads the environment, sets the offline and determinism variables, and forwards every argument. One submission covers one method, language, and scale across the declared tasks.

Repository Layout

safw/	the package
dexperts.py	the decoders: Proxy Tuning, TriMix, SAF-W
utils.py	cached batched generation and model loading
prompts.py	prompt builders for the six tasks
eval.py	accuracy, chrF++, and multilingual ROUGE-L
scripts/	
run.py	declaration driver and task table
run.sh	SBATCH wrapper for CSD3
infer.py	single-run inference engine
canonical.py	single source of truth for output paths
evaluate.py	scores one predictions file
eval.sh	idempotent scoring sweep over the results tree
get_perplexity.py	TriMix weight selection by perplexity
make_devx.py	builds the devx host-selection set
data/	MiLiC-Eval splits per task and language
results/	
devx/	host-selection scores behind the paper table
test_outputs/	canonical tree of predictions and metrics
analysis/	the analysis notebook, csv sources, and report figures
exploration/	
records/	development-time diagnostics, kept for provenance
report/	the compiled project report and executive summary
docs/	Sphinx documentation
tests/	unit tests
runpod/	environment setup for ad-hoc GPU pods
Dockerfile	CUDA runtime image with the ROUGE assertion

Paper-facing numbers live only under `results/`, and the report figures with their producer live in `analysis/`. The material under `exploration/records/` documents the development process, including the host-selection ablations discussed in the report.

Testing

Twenty-three tests cover the SAF-W fusion rule, the uniform-averaging reduction, the first-token anchor, and the declaration checks of the driver, including three accepted configurations and twelve refusals. Run them with `pytest tests/ -q`. The GitLab CI runs the same suite on every push.

Documentation

Install the extras and build with `pip install -e ".[docs]"` and `python -m sphinx -b html docs docs/_build/html`, then open `docs/_build/html/index.html`. The pages cover the method with full equations, installation, a quickstart, the reproduction guide, the data layout, and the API reference generated from the docstrings.

Use of Auto-generation Tools

Claude (Anthropic) assisted with code, experiment orchestration, debugging, and report drafting. GitHub Copilot provided occasional inline completions for docstrings and comments. Every generated artefact was reviewed, tested, and verified by the author. The full declaration, itemised by tool and activity, is in Appendix B.

Project Status

The pipeline is complete. The results tree is populated from the CSD3 runs. Audit the remaining unfilled files with the command in the Results section above.

Support, License, Authors

For questions, contact `tg561@cam.ac.uk`. The project is under the MIT License. Tianshuai Gao, Sidney Sussex College, University of Cambridge, MPhil in Data Intensive Science, supervised by Dr Weiwei Sun. The decoding framework extends the released TriMix codebase, the CPT checkpoints follow the TriMix release, evaluation uses the MiLiC-Eval benchmark, and experiments ran on the Cambridge Service for Data Driven Discovery (CSD3).